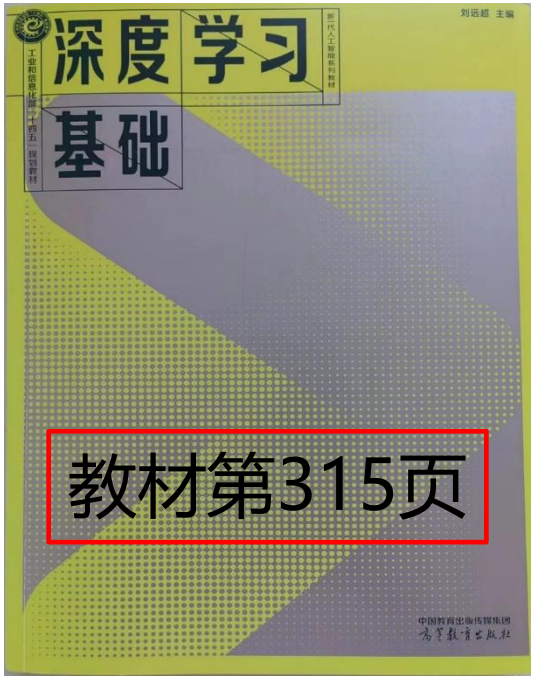
从强化学习到深度强化学习

有约不来过夜半，闲敲棋子落灯花。——宋·赵师秀

哈尔滨工业大学计算学部
刘远超
(ycliuharbin@163.com)

从强化学习到深度强化学习

本章在《深度学习基础》教材中的位置





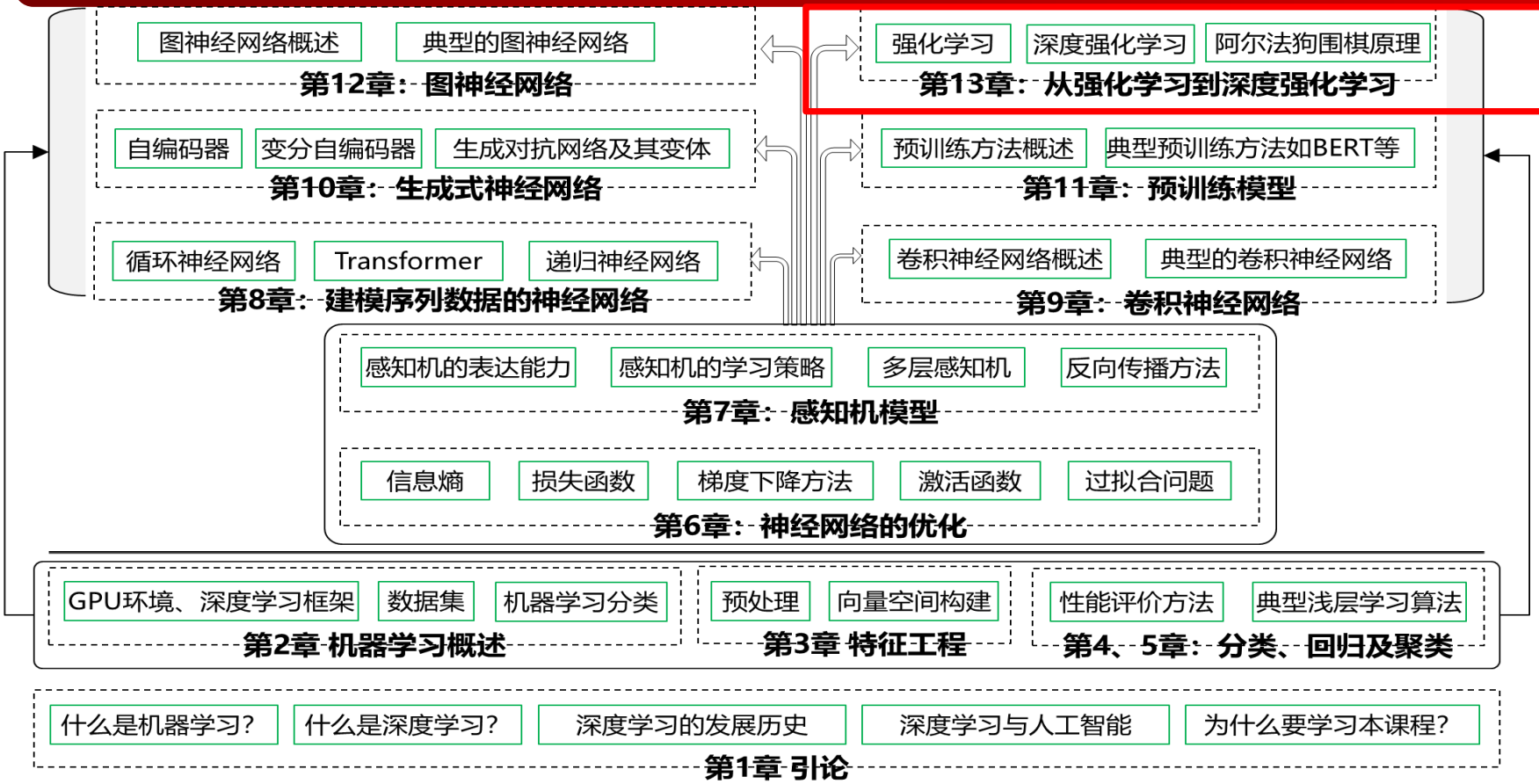


主编：<u>刘远超</u>；出版社：<u>高等教育出版社</u>；出版时间：<u>2023年09月20日</u>。

该教材先后入选：1）工信部十四五规划教材；2）国家新一代人工智能战略咨询委员会推出的新一代人工智能系列教材；3）智能基座精品教材

从强化学习到深度强化学习

本章内容在”深度学习之树”知识体系中的位置



主编：刘远超 出版社:高等教育出版社；出版时间:2023年09月。

该教材先后入选：1）工信部十四五规划教材；2）国家新一代人工智能战略咨询委员会推出的新一代人工智能系列教材；3）智能基座精品教材

从强化学习到深度强化学习

本章内容简介

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

从强化学习到深度强化学习

本章内容概要

强化学习

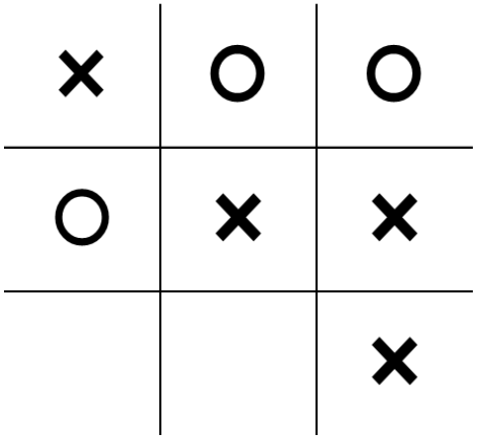
- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

强化学习的引入

从一个简单实例看什么是强化学习



三连棋游戏

- **游戏规则**是，双方轮流在一个3*3的九格方盘上划“✖”或者“○”，谁先把自己的同一符号排成横线、竖线、或者斜线，即赢得比赛游戏结束。如果九宫格填满也没有人赢，则和棋。
- 上述例子中，我们假设是由人类玩家与游戏程序交互，但也可以考虑设计一个虚拟的机器玩家，**即强化学习中所谓的智能体（Agent）**。

- **强化学习（Reinforcement Learning，RL）**：又称增强学习，是机器学习的范式和方法论之一，用于描述和解决智能体（agent）在与环境的交互过程中通过学习策略以达成回报最大化或实现特定目标的问题[1]．强化学习理论受到行为主义心理学启发，侧重**在线学习**通过接收环境对动作的奖励（反馈）获得学习信息并更新模型参数 。

[1] 邱锡鹏 著，神经网络与深度学习，第十四章 深度强化学习．Github Inc.．2019[引用日期2023-05-18]

从强化学习到深度强化学习

本章内容概要

强化学习

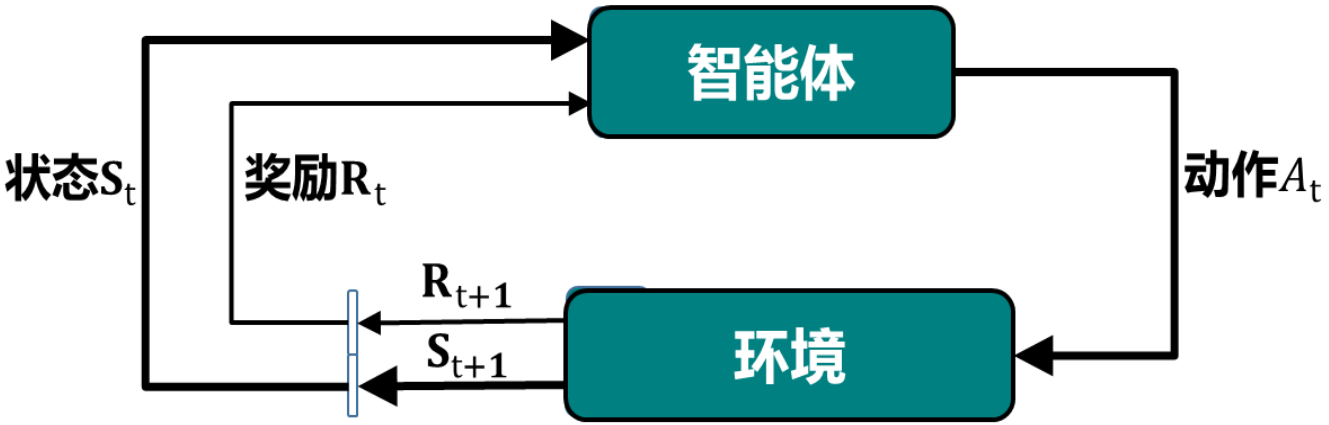
- 强化学习的引入
- **强化学习的要素**
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

强化学习的要素

要素体现了强化学习中智能体与环境的基本交互

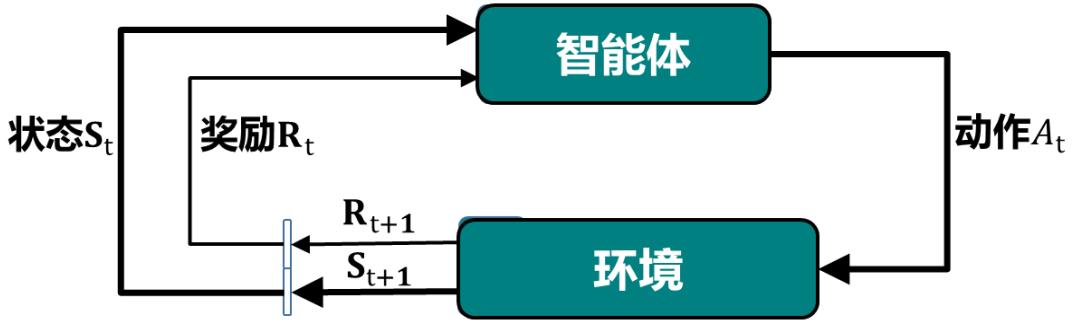


可以看到强化学习主要包含如下要素：

- t时间步的环境状态 S_t。
- t时间步智能体得到的奖励R_t。
- t时间步智能体所采取的动作A_t。

强化学习是与有监督学习、无监督学习相并列的机器学习范式

强化学习有何特点?



强化学习中智能体与环境的基本交互

- **试错学习**：强化学习需要训练对象和环境进行多次交互，通过试错的方式去总结出每一步的最佳行为决策，**整个过程没有指导，只有反馈（奖励）**。
- **延迟反馈**：有时可能要等到整个训练结束后才会得到一个反馈，比如Game Over或者是Win；
- **当前的动作影响之后收到的样本数据**：在当前状态下采取的动作，会影响下一步的状态，样本数据之间存在一定的关联性。但在有监督学习中，是假设训练样本之间独立的。

从强化学习到深度强化学习

本章内容概要

强化学习

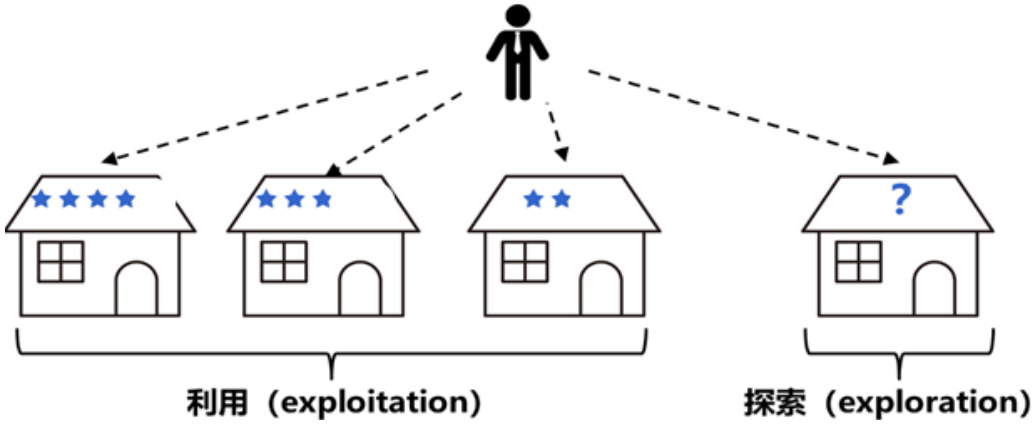
- 强化学习的引入
- 强化学习的要素
- **探索-利用困境**
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

探索-利用困境（Exploration-Exploitation Dilemma ）问题

什么是探索-利用困境？如何应对这个问题？



探索-利用困境问题举例

为了应对探索-利用困境问题，强化学习要考虑算法层面的模型要素：

1. **智能体采取动作所依据的策略（policy）π。**例如可以将状态为s时智能体采取的动作 a 表示为一个条件概率分布 $\pi(a|s) = P(A_t = a|S_t = s)$ 。
2. 智能体观察到某个具体的环境状态s后依据策略π采取某个具体行动a后的**综合动作价值函数$q_\pi(s, a)$。**

多选题 1分

设置

12

强化学习的要素包括哪些?

A 环境状态

B 智能体得到的奖励

C 智能体完成动作所用的时间

D 智能体所采取的动作

提交

多选题 1分

设置

13

下列关于强化学习与有监督学习的描述哪些是正确的?

A 强化学习也是一种有监督学习，每个样本都有标签。

B 强化学习需要考虑探索-利用困境问题。

C 在有监督学习中，假设训练样本之间独立的。

D 强化学习需要训练对象和环境进行多次交互，通过试错的方式去总结出每一步的最佳行为决策。

提交

主观题 10分

设置

14

举一个现实世界中探索-利用困境的例子，并说明你是如何解决的?

作答

从强化学习到深度强化学习

本章内容概要

强化学习

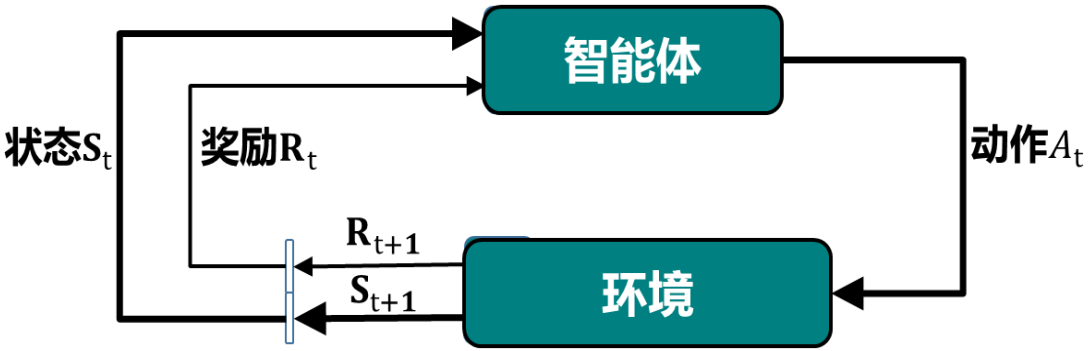
- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- **问题的简化：马尔可夫决策过程与贝尔曼方程**
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

强化学习问题的简化

马尔科夫决策过程



强化学习中智能体与环境的基本交互

- **马尔科夫决策过程的基本简化思想**是假设状态转化的马尔科夫性，即假设转化到下一个状态S_{t+1}的概率仅与上一个状态S_t有关，而与之前的状态无关。
- 同理，**策略π的马尔科夫假设**是假设在状态s时采取动作a的概率仅与当前状态s有关，而与其他的要素无关，即$\pi(a|s) = P(A_t = a|S_t = s)$。

那么，基于以上马尔科夫决策过程假设，**动作价值**和**状态价值**如何评估？

强化学习问题的简化：贝尔曼方程

1）动作价值函数的贝尔曼方程

动作价值函数的贝尔曼方程估计的是某个状态下采取某种动作的价值期望。

设$\mathbb{E}_\pi$表示期望，G_t表示考虑**折扣的收获**（discounted gain），是从状态S_t开始直到终止状态时所有奖励的衰减之和，γ∈[0, 1]是奖励衰减因子。

- **综合动作价值函数$q_\pi(s,a)$**不但要考虑当前动作的延时奖励R_{t+1}，还要考虑到后续其它时刻直到游戏结束的延时奖励，则$q_\pi(s,a)$可以表示为

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}_\pi(G_t|S_t=s,A_t=a) \\ &= \mathbb{E}_\pi(R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\ldots|S_t=s,A_t=a) \\ &= \mathbb{E}_\pi[R_{t+1}+\gamma(R_{t+2}+\gamma R_{t+3}+\cdots)|S_t=s,A_t=a] \\ &= \mathbb{E}_\pi[R_{t+1}+\gamma G_{t+1}|S_t=s,A_t=a] \\ &= \mathbb{E}_\pi(R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})|S_t=s,A_t=a) \end{aligned}$$

- 从而得到在当前状态s下采取动作a的动作价值函数$q_\pi(s,a)$的贝尔曼方程:

$$q_\pi(s,a)=\mathbb{E}_\pi(R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})|S_t=s,A_t=a)$$

其表明，t时刻的动作价值函数$q_\pi(S_t=s,A_t=a)$可以计算为其收到的奖励R_{t+1}与下一状态的动作价值收益期望$q_\pi(S_{t+1},A_{t+1})$的折扣的和。

强化学习问题的简化：贝尔曼方程

2）状态价值函数的贝尔曼方程

状态价值函数的贝尔曼方程估计的是某个状态下所有动作的价值期望。

- 评价当前整体状态（例如棋局状态）的状态价值函数$v_\pi(s)$：

$$v_\pi(s) = \mathbb{E}_\pi(G_t|S_t = s) = \mathbb{E}_\pi(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots|S_t = s)$$

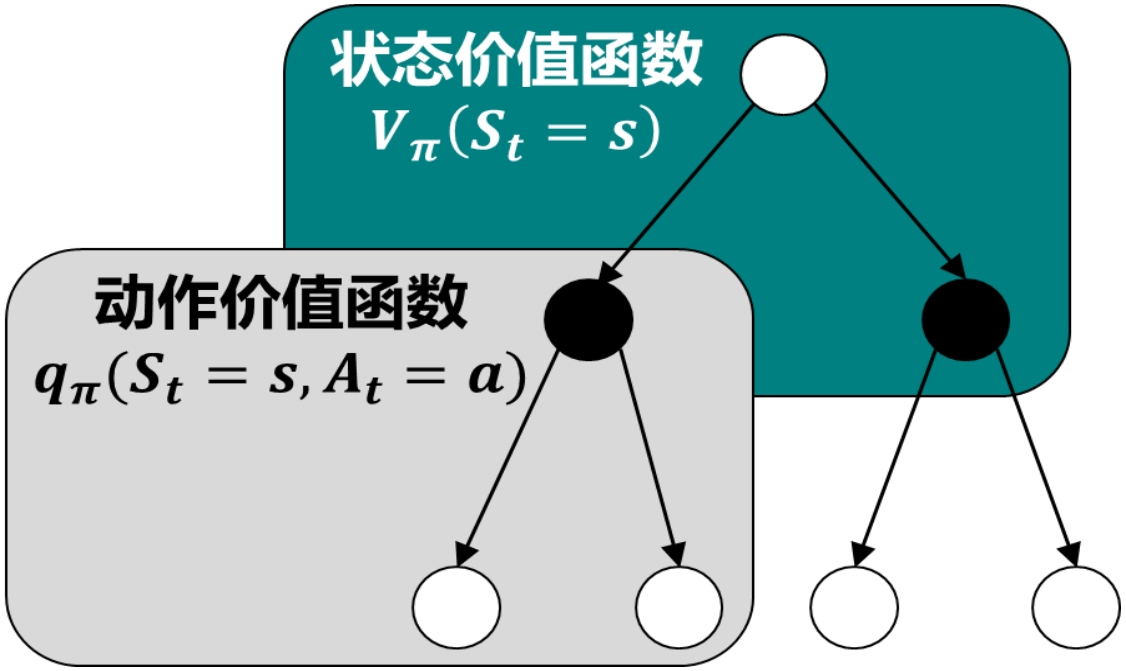
- 参照之前的推理过程，有$G_t = R_{t+1} + \gamma v_\pi(S_{t+1})$，则$v_\pi(s)$类似地也可以进一步改写为如下形式：

$$\boldsymbol{V_\pi(S_t = s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s]}$$

其表明，t时刻的状态价值函数$V_\pi(S_t = s)$可以计算为其收到的奖励R_{t+1}与下一状态的收益期望 $V_\pi(S_{t+1})$的折扣的和。

强化学习问题的简化：贝尔曼方程

状态价值函数和动作价值函数之间的区别



友情提示：要注意其中符号命名的区别

从强化学习到深度强化学习

本章内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- **序贯决策问题与强化学习**
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

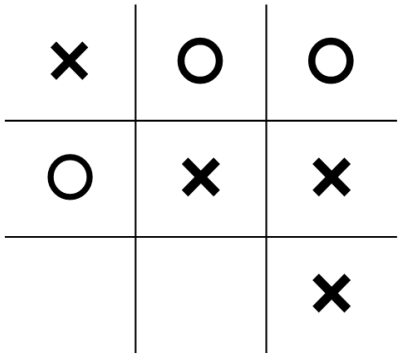
深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

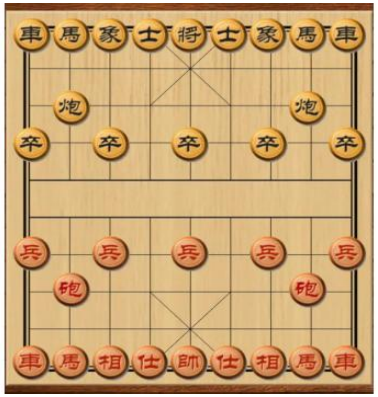
序贯决策问题与强化学习

序贯决策问题及其广泛的应用领域

- 所谓**序贯决策问题**（Sequential Decision Problem），又称为顺序决策或序列决策问题，本质上是一种随机性或不确定性动态系统的最优化问题，需要按时间顺序进行一系列动态决策。



三连棋



中国象棋



国际象棋

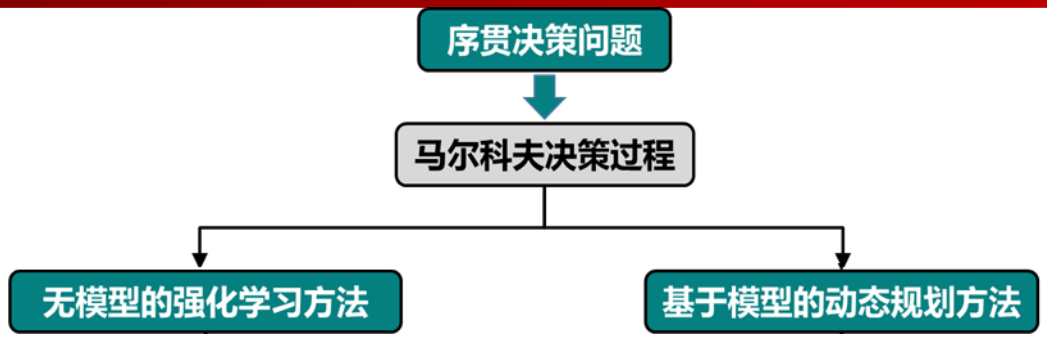


围棋

- 序贯决策广泛应用于现实世界中物流配送车辆调度、家电产品运营、应急资源配置等情况。
- **问题：你还能想出哪些序贯决策类问题?**

序贯决策问题与强化学习

序贯决策问题的求解方法图谱



一般通过马尔科夫决策过程及贝尔曼方程对问题进行简化。在此基础上，通常有两大类求解方法：

- **无模型（Model-Free）方法：**
  - **特点是不需要学习环境模型**。智能体只需要进行一个过程：基于当前状态和奖励进行决策，更易于实现和运行。
  - **典型方法**：Q-learning、SARSA、Deep Q-Network（DQN）。典型应用包括游戏智能体如雅达利（Atari）游戏、Flappy Bird等。
- **有模型（Model-based）方法：**
  - **特点是智能体在学习过程中对环境建模**，即学习环境的动态转移函数（也称为状态转移函数或转移模型），一般利用梯度下降方法求解最优决策策略。
  - **典型方法**：基于策略的梯度下降（Policy Gradient）、Actor-Critic等。典型应用包括AlphaGo和AlphaZero等棋类游戏。

从强化学习到深度强化学习

本章内容概要

强化学习

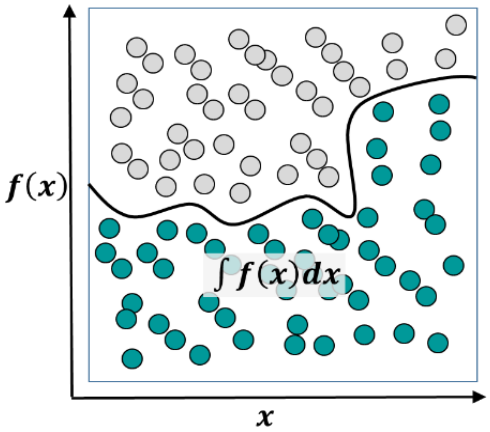
- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- **使用蒙特卡罗法求解强化学习**
- 时序差分法（TD）

深度强化学习

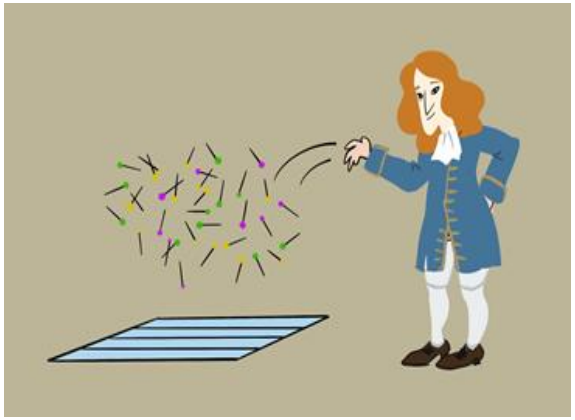
- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

强化学习中如何使用蒙特卡罗法求解序贯决策问题

什么是蒙特卡罗法（Monte Carlo method）



如何求曲线下的面积？



蒲丰（Buffon）投针计算圆周率

投针试验的历史资料（a 折算为1）

试验者	年份	针长	投针次数 n	相交次数 k	π 的试验值
Wolf	1850	0.8	5000	2532	3.1596
Smith	1855	0.6	3204	1219	3.1554
De.Morgan	1860	1	600	383	3.137
Fox	1884	0.75	1030	489	3.1595
Lazzerini	1901	0.83	3408	1801	3.1415929
Reina	1925	0.54	2520	859	3.1795

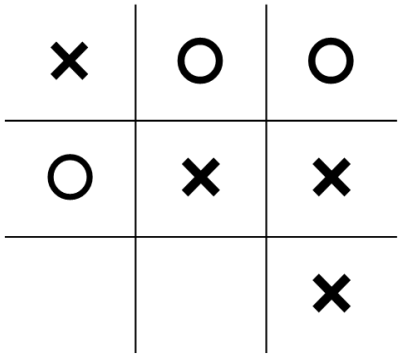
蒙特卡罗法也称统计模拟法、统计试验法。其原理是：

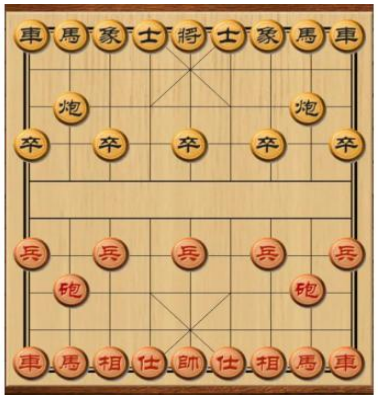
1）针对难以用解析等方法求解的实际问题**建立一个概率统计模型**，使所求实际问题的解恰好是该概率模型的某个量化数值特征；

2）通过大量足够的随机样本在计算机上模拟试验，按照抽样调查法**求取统计值来推定该实际问题的某个未知特性量的近似解**。

基于蒙特卡罗法求解序贯决策问题的具体方法（即策略评估）

如何应用蒙特卡罗方法？其适用条件是什么？









- 强化学习中的蒙特卡罗求解方法仅仅需要经验就可以估计状态或者动作的价值，这种**经验可以在线获得或者根据某种模拟机制获得**。
- **所谓经验就是训练样本**，通常需要由智能体与环境多次交互获得。**这样获得的一个序列数据就是训练样本。**
- 蒙特卡罗方法就是利用大量类似这样的样本，来估计状态s下遵循策略π的期望回报，即状态值函数$v_\pi(s)$，或者动作价值函数$q_\pi(s,a)$。
- **蒙特卡罗方法的适用条件**是其需要通过采样一定数量的经历完整的状态序列(episode)来估计状态的价值。

蒙特卡罗法求解强化学习预测问题的原理

如何通过蒙特卡罗方法来估算收获的期望?

对于某给定策略π，假设其采样获得的某个完整状态序列（设有T个状态）为：

$$S_1, A_1, R_2, S_2, A_2, \ldots S_t, A_t, R_{t+1}, \ldots R_T, S_T$$

- 根据前文所述，每个状态的综合收获G_t为其后续奖励的折扣求和，即

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \gamma^{T-t-1} R_T$$

- 且任意某个具体状态s的价值函数$v_\pi(s)$定义为

$$v_\pi(s) = \mathbb{E}_\pi(G_t | S_t = s)$$

即每个状态的价值等于该状态的综合收获的期望。

这个期望值可以通过蒙特卡罗方法来估算。即对所有**到达过该状态的收获取平均值来估计$v_\pi(s)$**（利用经验平均代替随机变量的期望）。其数学描述如下：

$$v_\pi(s) \approx average(G_t), s.t. S_t = s$$

主观题 10分

设置

27

结合下棋或者其它现实世界中的实例，你是如何理解每个状态的综合收获G_t为其后续奖励的折扣求和的?

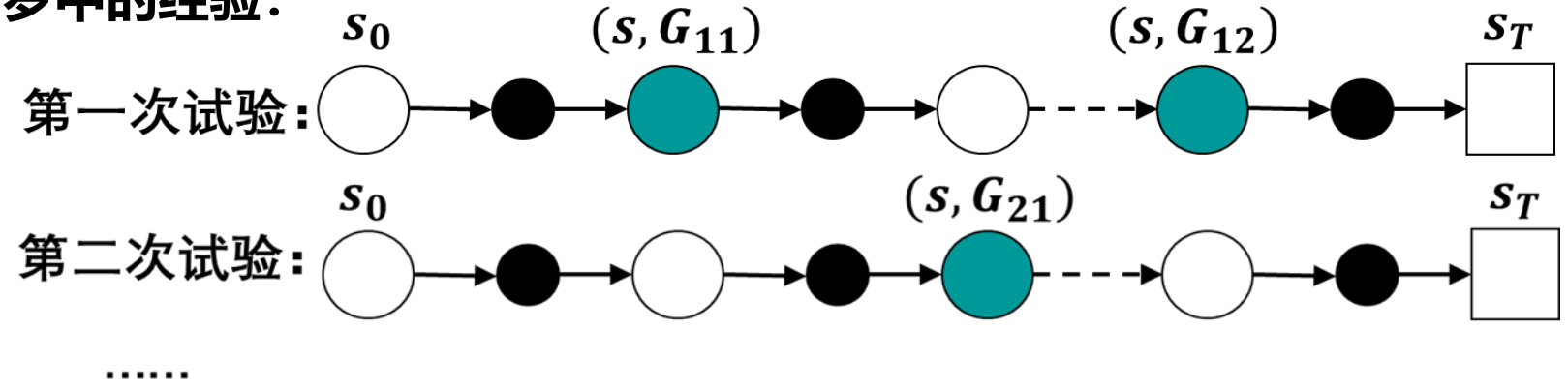
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \gamma^{T-t-1} R_T$$

作答

蒙特卡罗法求解强化学习预测问题的原理

举例：如何利用公式估算$v_\pi(s)$

蒙特卡罗中的经验：



由于同一状态可能在某个序列中出现多次，因此又分为首次访问（first visit）求均值方法和每次访问（every visit）方法求均值两种处理方法：

1）首次访问求均值方法：$v(s) = \frac{G_{11}(s)+G_{21}(s)+\cdots}{N(s)}$

2）每次访问求均值方法：$v(s) = \frac{G_{11}(s)+G_{12}(s)+\cdots+G_{21}(s)+\cdots}{N(s)}$

当采样获得的序列样本数量足够多时，使用蒙特卡洛方法获得的模拟近似值将趋近真实价值，即 $v(s) \to v_\pi(s)$ **as** $N(s) \to \infty$。

蒙特卡罗法求解强化学习预测问题的原理

如何采用累进更新平均值（incremental mean)迭代计算均值

- 每得到一个数据计算一次均值：在计算k次的均值μ_k时，利用上一次的均值μ_{k-1}、当前值x_k以及统计频次k即可。其推理过程如下：

$$\mu_k = \frac{1}{k}\sum_{j=1}^{k} x_j = \frac{1}{k}\left(x_k + \sum_{j=1}^{k-1} x_j\right) = \frac{1}{k}(x_k + (k-1)\mu_{k-1})$$

$$= \boldsymbol{\mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})}$$

- 因此，之前求状态价值估计值的蒙特卡罗方法中的平均值计算方法可以采用这种累进（增量）更新平均值的办法来计算：

$$N(s) = N(s) + 1$$

$$\boldsymbol{v(s) = v(s) + \frac{1}{N(s)}(G_t - v(s))}$$

其中G_t为满足约束为$S_t = s$条件的收获。

同理，也可以用蒙特卡罗方法求解动作价值函数$q_\pi(s,a)$的近似解$q(s,a)$：

蒙特卡罗法求解序贯决策问题的算法流程

如何应对探索-利用困境？ ϵ−贪心法选择策略

- 出于**探索-利用困境**的考虑，蒙特卡罗法训练过程在状态s下具有最大动作价值的动作$a^* = arg\ \underset{a \in A}{max}\ q(s,a)$会以很大的概率被选中执行。

- 蒙特卡罗法还要以一个较小的ϵ概率值从所有m 个动作中随机选择一个动作以进行适当的探索。这就是所谓的**ϵ−贪心法选择策略**。可以描述如下：

$$\pi(a|s) = \begin{cases} \frac{\epsilon}{m} + 1 - \epsilon & if\ \ a^* = arg\ \underset{a \in A}{max}\ q(s,a) \\ \frac{\epsilon}{m} & else \end{cases}$$

- **训练开始的阶段应该鼓励探索（Exploration）**；而在中后期，则应该逐渐侧重于利用。因此ϵ的取值一般多次迭代的过程中逐渐减小并趋于0。

基于以上讨论和知识铺垫，下面给出利用蒙特卡罗法求解强化学习控制问题的算法流程（以使用每次访问方法求均值为例）。

算法 13. 1: 蒙特卡罗法求解强化学习控制问题

Input: 状态集 S、动作集 A、衰减因子 γ、探索率 ε

Output: 最优动作价值函数 q^* 和最优策略 π^*

初始化: 所有的动作价值 $q(s,a)=0$，状态次数 $N(s,a)=0$，采样次数 $k=0$，随机初始化策略 $\pi(a|s)$, 蒙特卡罗采样次数 $k=0$

while 如果所有的 $q(s,a)$ 不收敛 **do**

  $k \leftarrow k+1$

  基于策略 π 进行第 k 次蒙特卡罗采样，得到一个完整的状态序列（episode）：S_1, A_1, R_2, ..., S_t, A_t, R_{t+1}, ... , R_T, S_T（其中任意 $S_t \in S$, $A_t \in A$）

  for each pair (s,a) appearing in the episode k **do**

    $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... \gamma^{T-t-1} R_T$

    $N(s,a) = N(s,a) + 1$

    $q(s,a) = q(s,a) + \frac{1}{N(s,a)}(G_t - q(s,a))$

  end

  for each s in the episode k **do**

    $\varepsilon = 1/k$

$$\pi(a|s) = \begin{cases} \varepsilon/m + 1 - \varepsilon & if\ a^* = \underset{a \in A}{\arg\max}\, q(s,a) \\ \varepsilon/m & else \end{cases}$$

  end

end

最终得到的所有 $q(s,a)$ 为最优动作价值函数 q^*，对应的策略 $(a|s)$ 为最优策略 π^*

主观题 10分

设置

32

请复述蒙特卡洛法求解强化学习控制问题算法的基本过程

作答

从强化学习到深度强化学习

本章内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- **时序差分法（TD）**

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

如何基于时序差分法（TD）求解序贯决策问题

对经历不完整的状态序列如何求解（估计状态和动作价值）？

- 如果在采样得到的训练样本中没有终止状态，则难以使用上述蒙特卡罗法。然而根据之前介绍的计算状态价值的**贝尔曼方程**，可以近似计算收获G_t：

$$G_t = R_{t+1} + \gamma v(S_{t+1})$$

- 参照参数更新的方法公式：新的估计值←原有估计值+StepSize*[目标值-原有估计值]，则状态价值$v(S_t)$的更新方法可以写为：

$$v(S_t) \leftarrow v(S_t) + \alpha[G_t - v(S_t)]$$

这里α为学习速率。由于差分法中无法使用完整状态序列样本，所以这里使用了一个常数系数$\alpha \in [0,1]$。

- 因此，结合以上两个公式，时序差分法中的状态价值和动作状态价值的迭代更新公式分别为：

状态价值：$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t))$ （13.20）

动作状态价值：$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - q(S_t, A_t)$ （13.21）

时序差分控制问题的分类

两类典型方法：SARSA算法和Q-Learning算法

状态价值：$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t))$ （13.20）

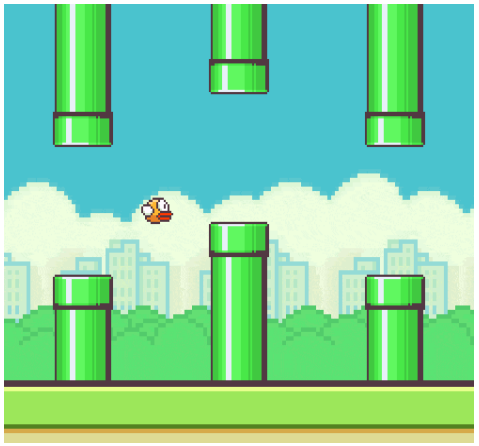
动作状态价值：$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha(R_{t+1} + \gamma q(S_{t+1}) - q(S_t, A_t)$ （13.21）

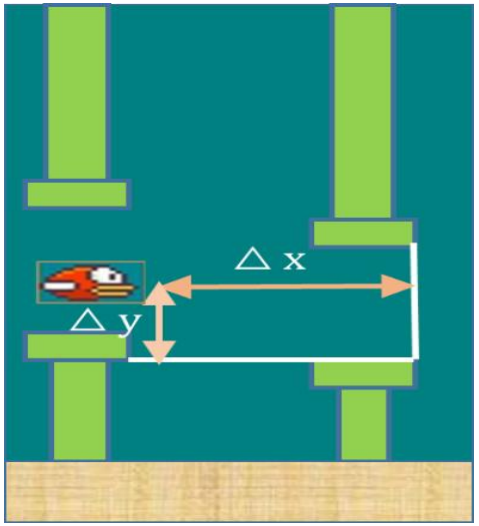
- 可以看出，时序差分法在更新状态价值时是基于即时奖励R_{t+1}和**下一状态S_{t+1}的预估价值**的和来近似估计当前状态S_t在状态序列结束时可能得到的综合收获，因此其是当前状态价值的有偏估计。
- 时序差分控制问题的求解方法又进一步分为在线控制（on-policy）和离线控制（off-policy）两类：
  1. 在线控制算法的特点是选择新动作的策略和更新价值函数的策略为**同一个**（例如都使用$\epsilon-$贪心法），**其典型代表为SARSA算法**；
  2. 离线控制一般使用两个策略，其中一个策略（例如$\epsilon-$贪心法）用于选择新的动作，另一个策略（例如贪心法）用于更新价值函数，**其典型代表为Q-Learning算法**。

时序差分在线控制算法SARSA

$State \rightarrow Action \rightarrow Reward \rightarrow State \rightarrow Action$

- SARSA算法名字来自于$State \rightarrow Action \rightarrow Reward \rightarrow State \rightarrow Action$（即状态、动作、奖励、状态、动作）中的首字母。
- 为便于介绍该算法，这里先介绍一款名为Flappy Bird（快乐飞行的小鸟）的简单游戏。
- SARSA算法训练的关键是通过若干次迭代更新**Q表（Q-table）**。Q表中包含所有状态及动作对的价值。





算法 13. 2: SARSA 训练算法

Input: 迭代轮数 T，状态集 S, 动作集 A, 步长 α，衰减因子 γ, 探索率 ϵ

Output: Q 表，其包含所有状态及动作对的价值 $q(s,a)$

随机初始化 $q(s,a)$

//开始新一轮游戏

for 对于每个回合（episode）**do**

  Initialize s //初始化起始状态，例如小鸟的初始位置

  以 s 为输入，利用 Q 表中的信息使用 ϵ− 贪心策略 π，得到动作 $a: a = \pi(s)$

  //对于当前回合中的每一步

  while s 不是终止状态（例如小鸟死亡）**do**

    （在游戏中）采取动作 a，观察到（游戏反馈的）的奖励 r 和（游戏反馈的）智能体的新状态（例如小鸟的新位置）s' //通常要实现一个游戏模拟环境，r 和 s' 由模拟游戏环境中给出

    以 s' 为输入，利用 Q 表中的信息使用 ϵ− 贪心策略 π，得到动作 $a': a' = \pi(s')$

    更新动作状态价值函数（即更新 Q 表中的某个信息项）

    $q(s,a) \leftarrow q(s,a) + \alpha * [r + \gamma * q(s',a') - q(s,a)]$

    更新位置（即状态）和动作：$s \leftarrow s'$；$a \leftarrow a'$

  end

end

主观题 10分

设置

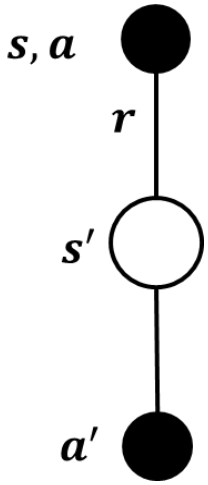
38

贪心法和ϵ−贪心法有何区别?

作答

时序差分在线控制算法SARSA

SARSA算法流程示意图



结合左图的SARSA算法流程示意图可见：

- 算法的每次迭代（即当前回合中的每一步），首先基于ϵ−贪心法在当前状态s选择并执行一个具体动作a，这样系统会转到一个新的状态s′，同时返回即时奖励r。
- 在新的状态s′，算法基于ϵ−贪心法再次选择一个动作a′，这个动作a′将被用来更新（通过查Q表）价值函数$q(s,a)$。
- 可见，一次更新需要用到5个数据，从时序上表示为$< S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1} >$，这也是这种方法命名为SARSA的原因。

- SARSA算法和即将介绍的Q-learning算法都是**基于Q表**。但和Q-Learning算法不同的是，SARSA算法在当前回合中的每一步得到状态s'时，会确定下一步将要采取的动作a'，**并在下一步真的采取了这个动作**（通过算法中第4步的更新状态和动作来实现），而不是在下一步开始时动态确定。

时序差分离线控制算法Q-Learning

与SARSA算法的主要不同在于其使用两个不同的控制策略

- Q-Learning算法也是一种使用时序差分求解强化学习控制问题的方法。
- 其与SARSA算法的主要不同在于其使用两个不同的控制策略：一个策略用于选择新的动作，另一个策略用于更新动作状态价值函数。因此其是时序差分中的离线控制算法。
- Q-Learning算法训练的关键也是通过若干次迭代更新Q表（Q-table）。Q表中包含所有状态及动作对的价值。

算法 13. 3: Q-learning 训练算法

Input: 迭代轮数 T，状态集 S, 动作集 A, 步长 α，衰减因子 γ, 探索率 ϵ

Output: Q 表，其包含所有状态及动作对的价值 $q(s,a)$

随机初始化 $q(s,a)$
//开始新一轮游戏
for 对于每个回合（episode）**do**
    Initialize s //初始化起始状态，例如小鸟的初始位置
    //对于当前回合中的每一步
    while s 不是终止状态（例如小鸟死亡）**do**
        以 s 为输入，利用 Q 表中的信息用 ϵ− 贪心法，得到动作 a：a=π(s)
        （在游戏中）采取动作 a，观察到（游戏反馈的）的奖励 r 和（游戏反馈的）智能体的新状态（例如小鸟的新位置）s' //通常要实现一个游戏模拟环境，r 和 s' 由模拟游戏环境中给出
        更新动作状态价值函数（即更新 Q 表中的某个信息项）

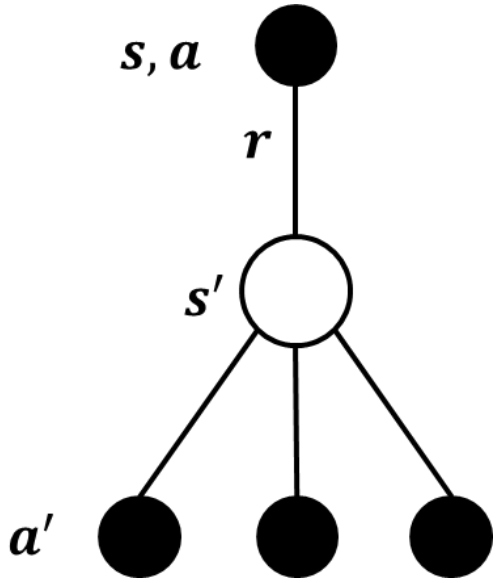
$$q(s,a) \leftarrow q(s,a) + \alpha * [r + \gamma * \max_{a'} q(s',a') - q(s,a)]$$

        更新状态 (例如小鸟的位置)：$s \leftarrow s'$
    end
end

$\max_{a'} \mathbf{q}(\mathbf{s}', \boldsymbol{a}')$是新状态$\boldsymbol{s}'$（例如小鸟的下一个位置，注意不是$\boldsymbol{s}$）下智能体采用某种可能动作所能给出的最大收益（仍到Q表中检索）。通过加上$\max_{a'} \mathbf{Q}(\mathbf{s}', \boldsymbol{a}')$，使得Q表中该条目即$\mathbf{q}(\mathbf{s}, \mathbf{a})$得以对未来进行了某种提前感知。

时序差分离线控制算法Q-Learning

Q-Learning算法的流程示意图



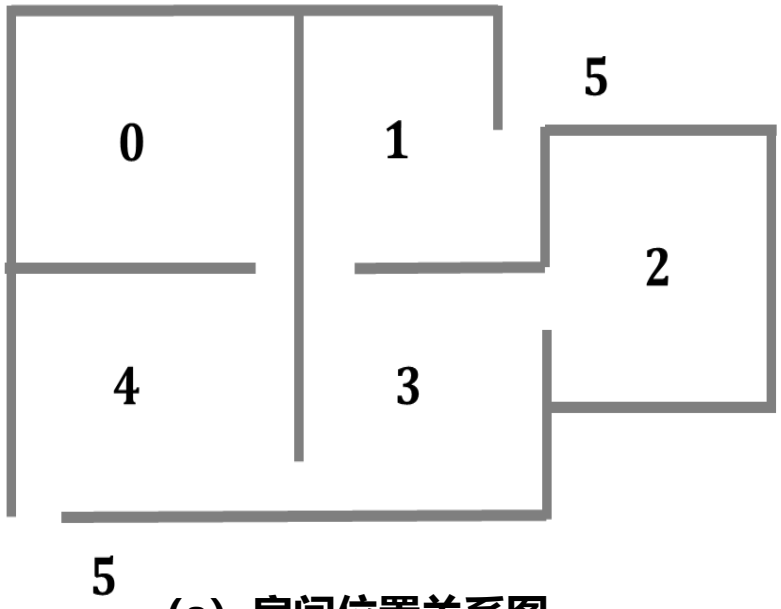
Q-Learning算法的流程示意图

- Q-Learning首先基于状态s，用ϵ−贪心法选择得到动作a 然后执行动作a，得到奖励r，并进入状态s′。
- 基于此处的状态s′，Q-Learning没有像SARSA那样使用ϵ−贪心法选择a′，**而是使用贪心法**选择a′，也就是说，选择使$q(s',a')$最大的a作为a′来更新价值函数，即算法更新公式中的$\max_{a'} q(s',a')$。

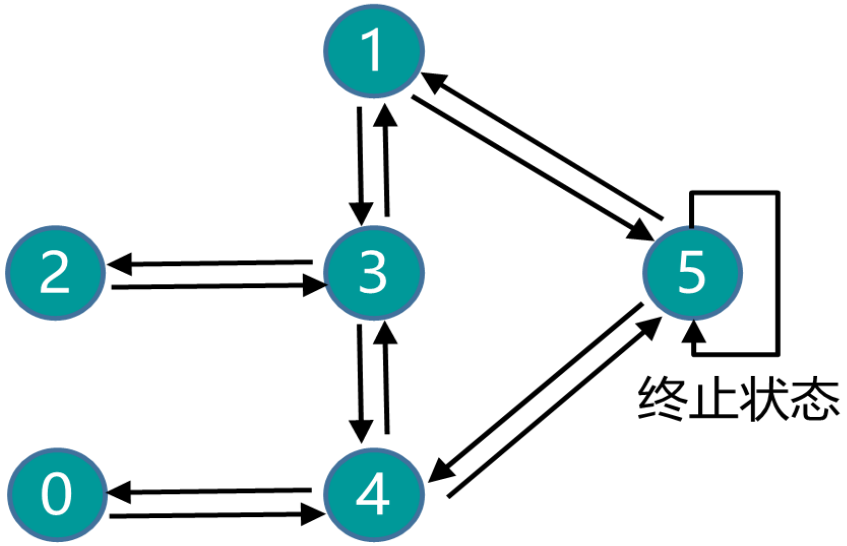
  相当于从左图中下方的三个黑色实心圆所代表的动作中选择一个$q(s',a')$最大的动作。
- 此时选择的动作只会参与动作价值函数的更新，不会真正被执行。动作价值函数更新后，进入下一时间步后，需要基于新得到的状态s′，用ϵ−贪心法重新选择新的动作并执行。这一点也和SARSA有所不同。

Q-Learning小例子

智能体走出迷宫小例子



(a) 房间位置关系图



(b) 抽象得到的节点（状态）关系图

求解过程：可以分为两个阶段：

1）训练阶段。构造并学习得到Q表，其用来存储智能体从经验中学到的知识；

2）测试阶段。Q表构造完毕后，智能体从任意房间出发，将利用Q表找到走到建筑外空地的最优路径。

Q-Learning小例子

先构造存储奖励值（reward）的矩阵R，如图。智能体走迷宫小例子中的Q-Learning算法的工作过程为：

1）**首先进行必要的初始化工作。**例如初始化折扣系数$\gamma = 0.8$，$\epsilon = 0.05$，$\alpha = 1.0$。构造Q表矩阵，其与R同阶，行表示状态，列表示动作。由于算法刚开始时智能体没有关于环境的知识，因此将Q表矩阵全零初始化。

2）假设智能体（agent）的初始状态为处于房间1中，根据Q-Learning算法“以s为输入，利用Q表矩阵中的信息用ϵ-贪心法，得到动作a：$a=\pi(s)$”的原则，智能体的下一个动作有0.95的概率从Q表中查询选择最大值得到，而有0.05的概率是随机从可选行为中选择得到。根据矩阵R的第二行（对应房间1或状态1）提示的连接信息，下一个房间只有两个选项：即状态（房间）3或状态（房间）5。且此时Q表中二者的动作价值均为0，则此时无论是哪种方式，都将从状态3或状态5中随机选择其一。

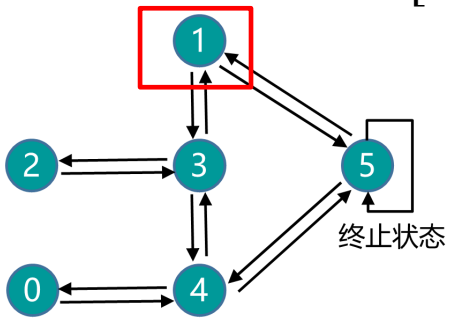
3）假设下一个动作是选中并进入状态5，然后根据公式更新动作价值函数：

$$q(s,a) \leftarrow q(s,a) + \alpha * \left[r + \gamma * \max_{a'} q(s',a') - q(s,a)\right]$$

由于是从状态1到状态5，因此需要根据该公式更新Q表中的$q(1,5)$条目：

$$q(1,5) = q(1,5) + \alpha * [r(1,5) + 0.8 * max\{q(5,1),\ q(5,4),\ q(5,5)\} - q(1,5)]$$

$$= 0 + 1.0 * [100 + 0.8 * 0 - 0] = 100。$$



R=

状态 \ 动作	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

奖励值矩阵R

Q=

状态 \ 动作	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

Q表的初始化

Q=

状态 \ 动作	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

Q表的第一次更新

Q-Learning小例子

Q表矩阵是如何更新的

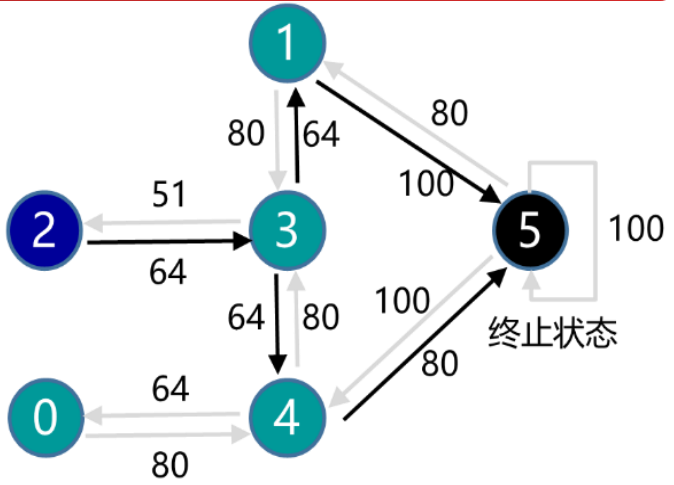
Q=

状态 \ 动作	0	1	2	3	4	5
0	0	0	0	0	400	0
1	0	0	0	320	0	500
2	0	0	0	320	0	0
3	0	400	256	0	400	0
4	320	0	0	320	0	500
5	0	400	0	0	400	500

Q=

状态 \ 动作	0	1	2	3	4	5
0	0	0	0	0	80	0
1	0	0	0	64	0	100
2	0	0	0	64	0	0
3	0	80	51	0	80	0
4	64	0	0	64	0	100
5	0	80	0	0	80	100

Q表矩阵的最终结果示例（右侧为规范化后结果）



（Q-Learning算法中）利用最终学得的Q表矩阵求解走出迷宫的路径

- 假设最终的Q表矩阵如**左侧图**所示，对其进行规范化，每个非零元素都除以Q表矩阵的最大元素，将得到**中间图**所示的结果。
- 通过使用Q-Learning算法，Q表矩阵接近收敛状态后，智能体便学习到了转移至目标状态的最佳路径（体现在Q表矩阵中）。例如**右图**给出了初始状态为2时的最佳路径。

Q-learning 和 SARSA的对比

通过Cliff Walking示例来看Q-learning 和 SARSA的差异

- 问题1：你能总结一下二者的异同吗?

相同点：

1）Q-learning 和 SARSA都是基于时序差分的强化学习方法；

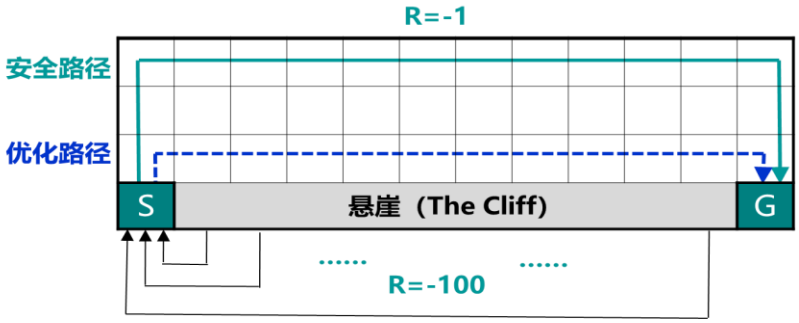
2）二者都使用评估策略更新Q值表格（即“学习”）。

不同点：

1）Q-learning算法 采用异策略（off-policy），即更新Q值表用的评估策略和实际行动采用的动作策略不同。其中**评估策略采用的是贪心策略**，而行动策略采用的是ε-贪心策略；

2）SARSA算法采用同策略（on policy），即更新Q值表用的评估策略和实际行动采用的动作策略相同，都为ε-贪心策略，都含有探索的因素。**因而SARSA算法是一种相对保守的算法。**

- 问题2：如下的Cliff Walking示例，猜一下两条路线分别是哪种方法？为什么?



智能体以网格的左下角位置为起点，右下角位置为终点，通过不断的移动到达右下角终点位置的问题。智能体每次可以在上、下、左、右这4个方向中移动一步，每移动一步便会得到-1单位的奖励。但是智能体在移动的过程中有以下的限制：(1)智能体不能移出网格，如果智能体想执行某个动作移出网格，那么这一步智能体不会移动，但是这个操作依然会得到-1单位的奖励；(2)如果智能体掉入悬崖，则会立即回到起点位置重新开始移动，并且会得到-100单位的奖励；(3)当智能体移动到终点时，该回合结束，该回合总奖励为各步奖励之和。

从强化学习到深度强化学习

本章内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- **价值函数的近似表示**
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

价值函数的近似表示

为何需要价值函数的近似表示?

Q=

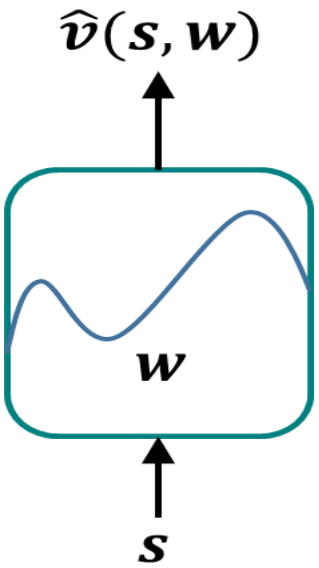
状态 \ 动作	0	1	2	3	4	5
0	0	0	0	0	80	0
1	0	0	0	64	0	100
2	0	0	0	64	0	0
3	0	80	51	0	80	0
4	64	0	0	64	0	100
5	0	80	0	0	80	100

Q表矩阵示例

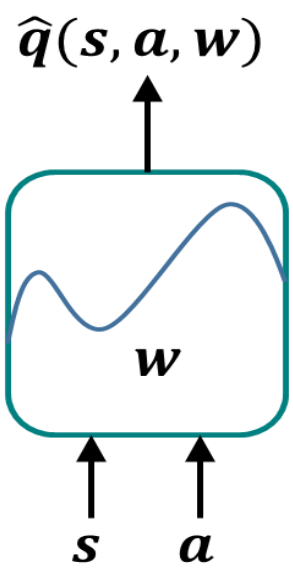
- 大数据时代的现实世界或者虚拟世界中的状态是**接近无限多的**（比如围棋），有时**甚至出现连续的**状态，此时传统方法将难以应对。
- 例如Q-Learning中的Q表可以通俗地被理解为一种备忘单（cheat sheet），但当状态和操作数目较多时Q表将变得很大，难以在内存中有效存储。
- 此时，一种可行的建模方法是引入一个**带有参数θ的近似函数，从而模拟价值函数**。理论上，这个近似函数的输入可以为任何连续或者离散数值。

价值函数的近似方法

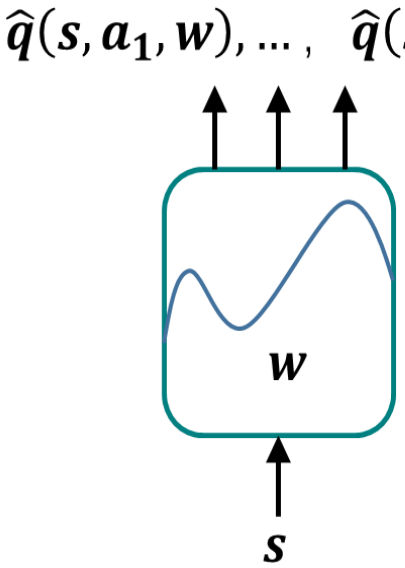
基于神经网络进行价值函数的近似表示是一种主流方法



状态价值函数近似方法



动作价值函数近似方法一

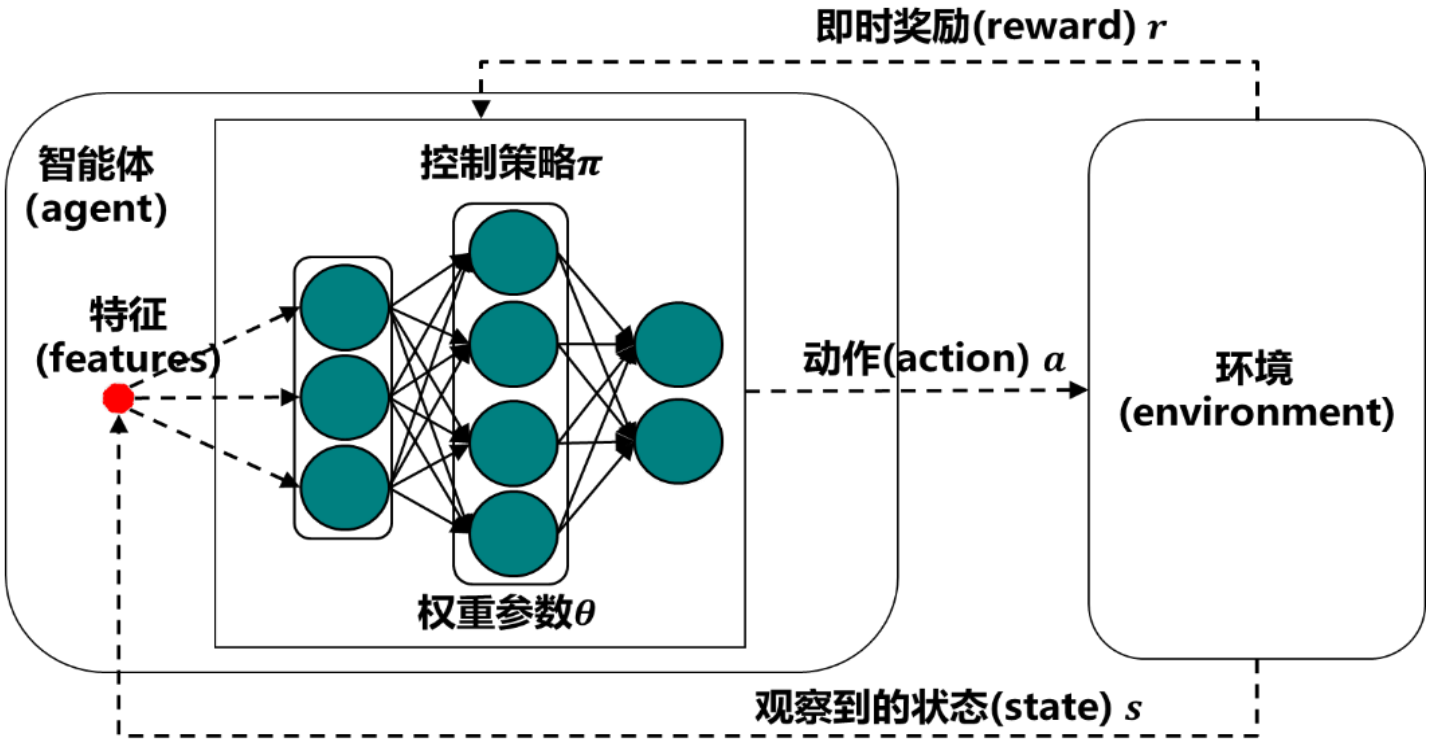


动作价值函数近似方法二

- 一种最简单的价值函数近似方法是线性变化法。假设ϕ(s)为状态s的特征向量，则状态价值函数可以近似为：$\hat{v}(s,w)=w^T\phi(s)$，其中w为权重参数。
- 而由于神经网络具有较强的拟合能力，可以自动提取复杂特征，因此基于神经网络和深度学习的方法实现价值函数的近似表示成为一种主流方法。此时**可以不必在表格中记录 Q 值，而是直接使用神经网络预测Q 值。**

什么是深度强化学习

深度强化学习使用神经网络来近似表示强化学习中的价值函数



深度强化学习使用神经网络来近似表示强化学习中的价值函数

从强化学习到深度强化学习

本章内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- **NIPS DQN**
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

NIPS DQN

NIPS DQN与Q-Learning相比的特点是什么?

- DQN（即Deep Q-Network）由 DeepMind于 2013年提出，发表在NIPS国际会议上，因此又称为NIPS DQN。
- NIPS DQN**首次将深度学习引入强化学习**中，因而被誉为是深度强化学习的开山之作。
- NIPS DQN延续了之前小节中介绍的Q-Learning的基本思路。相比之下，其特色包括如下几点：

  1）NIPS DQN中样本的Q值有两类：即预测得到的Q值和目标Q值（真实值）。其中预测的Q值是通过神经网络得到，即**Q网络**。Q网络的输入是状态s经过处理后得到的状态向量$\phi(s)$。Q网络主要是卷积神经网络CNN，但也可以是其它的网络结构。

NIPS DQN

NIPS DQN与Q-Learning相比的特点还有什么?

NIPS DQN的特点（续）：

2）NIPS DQN中Q网络训练样本的标签所采用的目标值（即**参考真值）仍然由智能体与环境交互得到并通过传统的Q-Learning方法进行计算**。然后通过Q网络来拟合函数设法使Q网络预测的Q值与Q值真值接近。当神经网络训练完成进入测试阶段后，智能体只使用Q网络计算Q值并进行路径决策。

3）NIPS DQN中Q网络中批次训练样本的获得是通过一种称之为经验回放（Experience Replay）的机制得到。下面对采用这种经验回放机制的原因及其具体原理进行介绍。

基于经验回放机制的数据采样

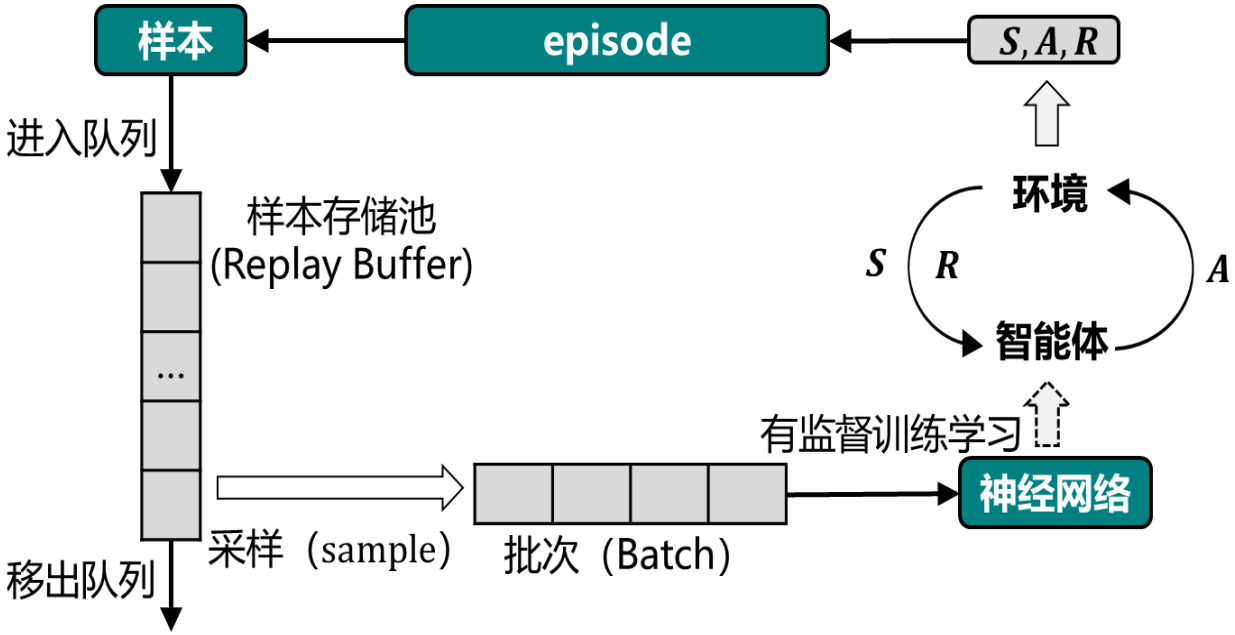
经验回放机制的作用是什么?

- **经验回放机制的动机**是：深度神经网络基于最大似然法，要求数据满足独立同分布假设。但传统Q Learning 算法中样本从游戏中连续帧依次获取，**前后之间关联性很强**（在简单的强化学习问题如三连棋中这个问题并不突出），可能使学习得到的值函数模型产生很大的波动。
- 经验回放的机制**包括收集样本和采样样本两个过程**：

1. 收集样本：构建一个经验回放池或称样本存储池（memory pool或Replay Buffer），按照时间先后顺序将样本存入。
2. 采样样本：每次训练从样本存储池中随机均匀采样数据（称之为经验）进行基于梯度下降的有监督学习。

NIPS DQN

经验回放机制示意图



- 经验回放机制示意图（S：State；A：Action；R：Reward）。其中智能体通过与环境交互源源不断地获取样本，送入到样本池中。
- 深度学习的神经网络在每个训练时从样本池随机均匀采样一个批次（batch）的样本。
- Q-Learning和深度学习的训练同步进行。

NIPS DQN算法的伪代码：

输入：迭代轮数（episode）T、状态特征维度n、动作集、 步长α、衰减因子γ、探索率ϵ、 Q网络结构、 批量梯度下降的样本数m

输出：Q网络参数θ

具体步骤：

1. 初始化一个容量为N的经验回放池$\mathcal{D}$
2. 随机初始化Q网络的参数 θ
3. for i= 1 to M：#进行每个状态序列或轮次（episode）的迭代
   1）初始化$s_1 = \{x_1\}$，即其为当前状态序列i的第一个状态（第一幅图像），得到预处理的序列$\emptyset_1 = \emptyset(s_1)$
   2) for t = 1 to T：#当前状态序列中智能体与环境的每一步交互
      a) 用ϵ−贪婪法得到动作a_t#此时动作的选择用到了Q网络
      b) （在游戏模拟器中）执行动作a_t，观察到奖励r_t和图像x_{t+1}（下一状态），以及是否是终止状态的标记 is_end_t
      c) 令$s_{t+1} = s_t, a_t, x_{t+1}$，预处理$\emptyset_{t+1} = \emptyset(s_{t+1})$
      d) 将元组$\{\emptyset_t, a_t, r_t, \emptyset_{t+1}, is_end_t\}$存入经验回放池$\mathcal{D}$·
      e) 从经验回放池$\mathcal{D}$中采样m个样本（即一个批量）$\{\emptyset_j, a_j, r_j, \emptyset_{j+1}, is_end_j\}, j = 1,2.,,,m$，计算每个样本的当前目标Q值（作为真实值）$y_j$：

$$y_j = \begin{cases} r_j & is_end_j\ is\ true \\ r_j + \gamma \max\limits_{a'} Q(\emptyset_{j+1}, a', \theta) & is_end_j\ is\ false \end{cases} \quad (13.28)$$

      a) 使用均方差损失函数$\frac{1}{m}\sum_{j=1}^{m}(y_j - Q(\emptyset_j, a_j; \theta))^2$，通过神经网络的梯度下降法和反向传播来更新Q网络的所有参数 θ
   3）end for
4. end for

从强化学习到深度强化学习

本章内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- **NATURE DQN**
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

Nature DQN

NIPS DQN的不足是什么?

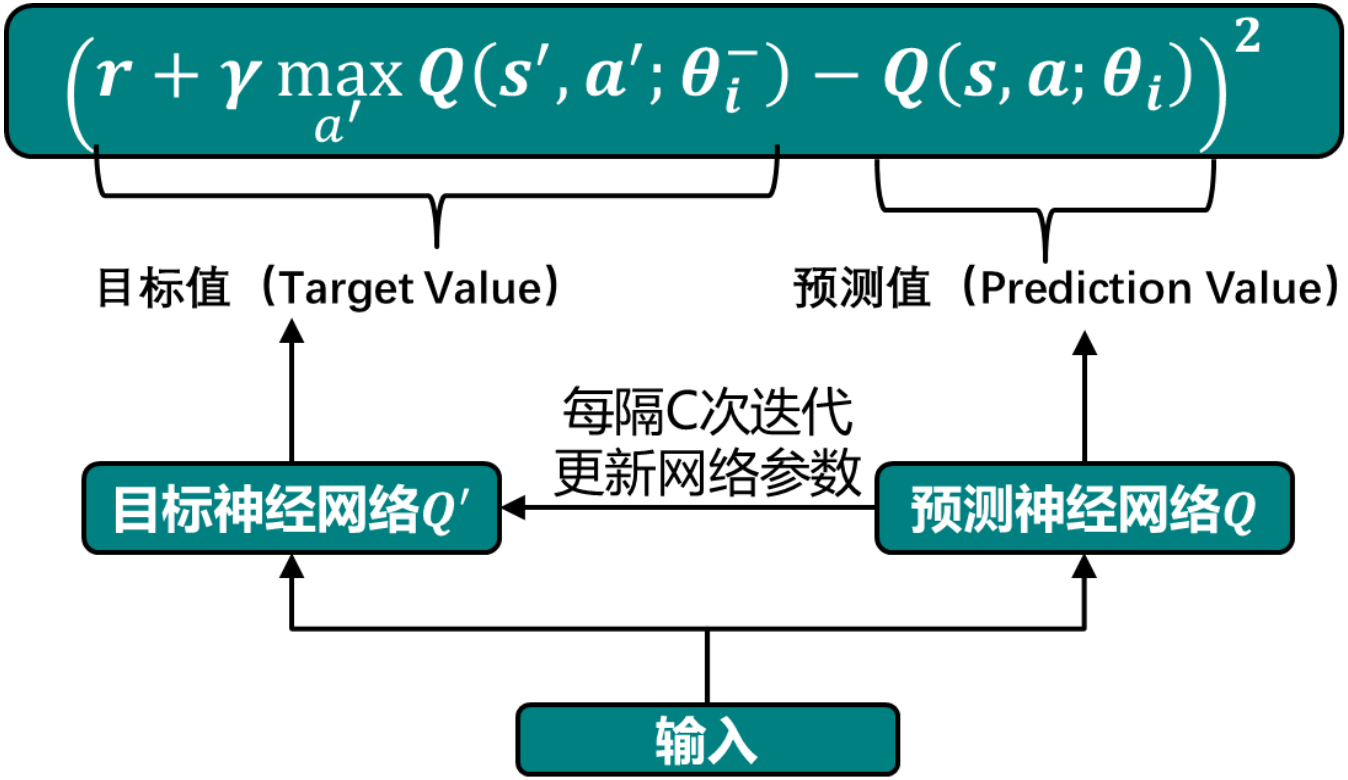
- 刚才提到在NIPS DQN算法中，所有的Q值计算都需要通过同一个Q网络计算得到。即有监督学习用到的Q值真值和Q值的预测值都是用同一个神经网络得到。在计算Q值真值（目标值）时用到了当前训练得到的Q网络参数。如该算法中最后y_j的这个计算公式（13.28）所示：

$$y_j = \begin{cases} r_j & is_end_j\ is\ \ true \\ r_j + \gamma \max\limits_{a'} Q(\emptyset_{j+1}, a', \theta) & is_end_j\ is\ false \end{cases}$$

- 因而这个Q值的基本真值（目标值）不是静态的，而是在不断变化。换句话说，其训练过程试图拟合总是变化的目标，其在学习映射不断变化的输入和输出。从而不利于算法的收敛。

Nature DQN

Nature DQN是NIPS DQN的改进版



- 因此，在其改进版即Nature DQN中在原有的Q网络的基础上额外增加一个Q网络$\hat{Q}$，称之为目标网络（Target Network）专门用于Q值真值（目标值）的计算。

下面仍以快乐的小鸟游戏为例，给出Nature DQN算法的伪代码：

输入： 迭代轮数T，状态特征维度n，动作集A，步长α，衰减因子γ，探索率ϵ，当前Q网络Q，目标Q网络$\hat{Q}$，批量梯度下降的样本数m，目标Q网络参数更新频率C。

输出： Q网络参数

具体步骤：

1. 初始化一个容量为N的经验回放池D
2. 随机初始化预测网络Q的参数θ
3. 随机初始化目标网络$\hat{Q}$的参数θ^-
4. for i= 1 to M：#进行每个状态序列或轮次（episode）的迭代
   1) 初始化$s_1 = \{x_1\}$，即其为当前状态序列i的第一个状态（第一幅图像），得到预处理的序列$\emptyset_1 = \emptyset(s_1)$
   2) for t = 1 to T：#当前状态序列中智能体与环境的每一步交互
      a) 用ϵ−贪婪法得到动作a_t：即以ϵ的概率随机选择得到动作a_t，否则选择 $a_t = \underset{a}{max} Q(\emptyset(s_t), a; \theta)$。#此时动作的选择用到了Q网络
      b) （在游戏模拟器中）执行动作a_t，观察到奖励r_t和图像x_{t+1}（下一状态），以及是否是终止状态的标记 is_end_t
      c) 令$s_{t+1} = s_t, a_t, x_{t+1}$，预处理$\emptyset_{t+1} = \emptyset(s_{t+1})$
      d) 将元组$\{\emptyset_t, a_t, r_t, \emptyset_{t+1}, is_end_t\}$存入经验回放池$\mathcal{D}$。
      e) 从经验回放池$\mathcal{D}$中采样m个样本（即一个批量）$\{\emptyset_j, a_j, r_j, \emptyset_{j+1}, is_end_j\}, j = 1,2.,,,m$，计算每个样本的当前目标Q值$y_j$：

$$y_j = \begin{cases} r_j & is_end_j \ is \ true \\ r_j + \gamma \underset{a'}{max} \hat{\boldsymbol{Q}}(\emptyset_{j+1}, \boldsymbol{a'}, \boldsymbol{\theta^-}) & is_end_j \ is \ false \end{cases} \quad (13.29)$$

      f) 使用均方差损失函数$\frac{1}{m}\sum_{j=1}^{m}(y_j - Q(\emptyset_j, a_j; \theta))^2$，通过神经网络的梯度下降法和反向传播来更新Q网络的所有参数 θ
      g) 每隔C个时间步，复制参数：$\boldsymbol{\theta^- = \theta}$

从强化学习到深度强化学习

本章内容概要

强化学习

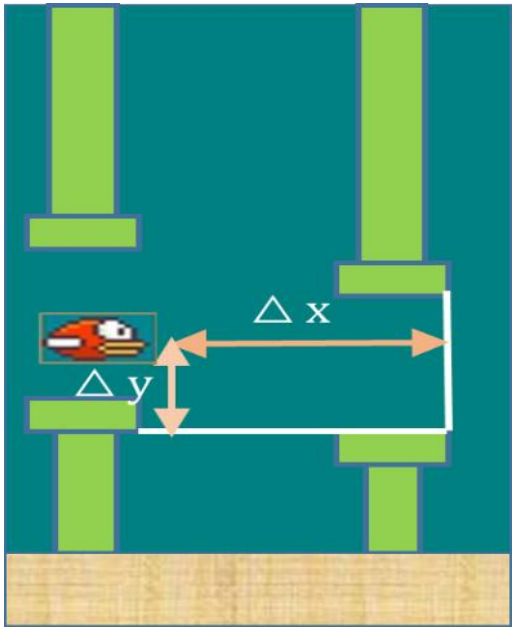
- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- **DQN典型应用案例FlappyBird**
- 基于深度强化学习的AI围棋模型AlphaGo Zero

DQN应用实例—Flappy Bird

使用传统的Q-learning方法训练Flappy Bird游戏中的智能体



状态	飞	不飞
$(\triangle x_1, \triangle y_1)$	1	20
$(\triangle x_1, \triangle y_2)$	20	-100
…	…	…
$(\triangle x_m, \triangle y_{n-1})$	-100	2
$(\triangle x_m, \triangle y_n)$	50	-200

- 其每一行记录了在状态（$\Delta x, \Delta y$）下，选择不同动作（飞或不飞）时的奖励。
- 在测试阶段，智能体（即小鸟）只需要根据当前位置（$\Delta x, \Delta y$）到Q表中查找到对应的行，选择收获值较大的动作作为当前状态下的动作即可。

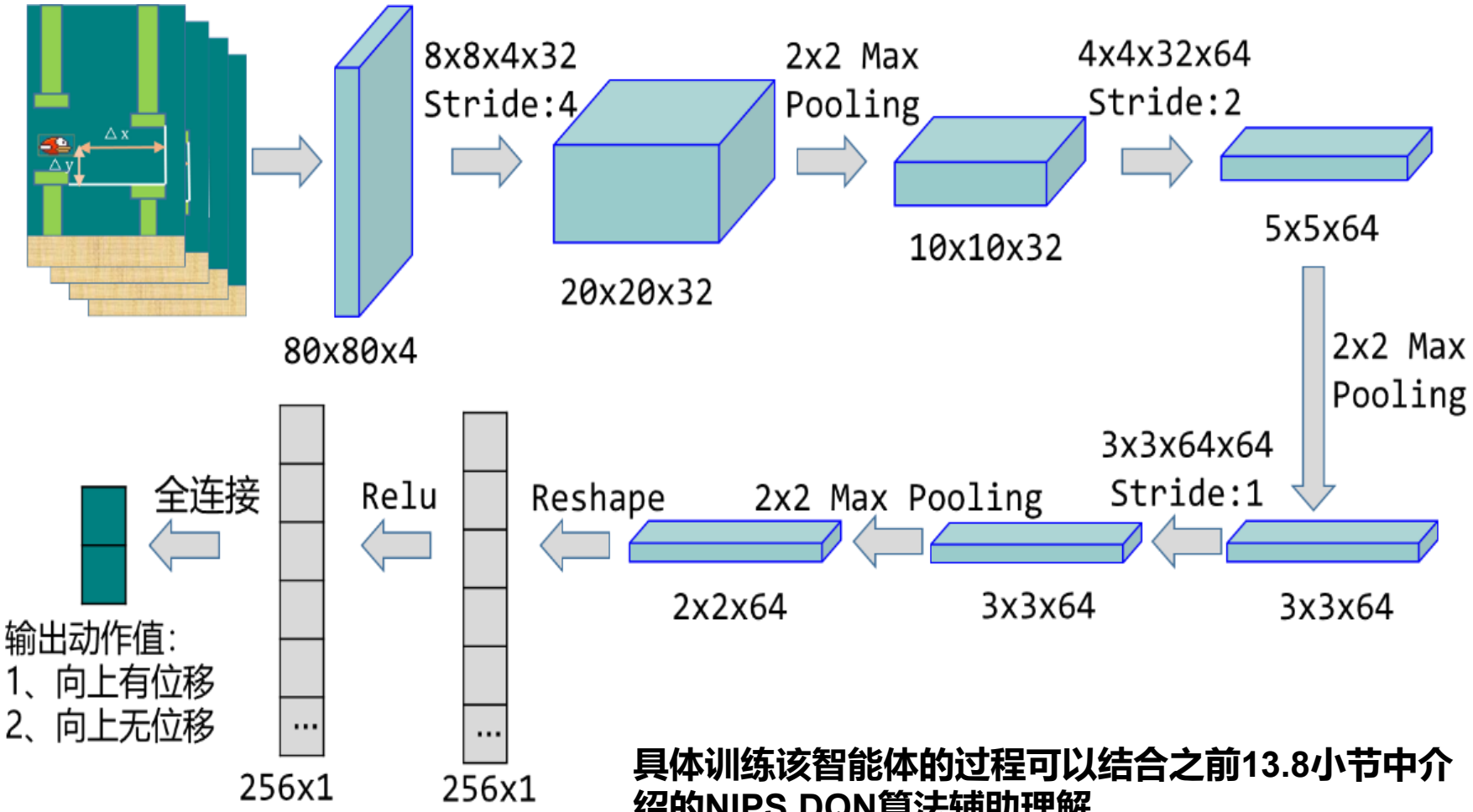
DQN应用实例—Flappy Bird

使用NIPS DQN来训练Flappy Bird游戏中的智能体

- NIPS DQN方法 曾经在Atari系列游戏上取得了很好的效果。使用NIPS DQN方法训练Flappy Bird游戏智能体的思路与开发Atari系列游戏的智能体类似。
- 结合之前小节中介绍的NIPS DQN算法，可知其特点是：不再基于小鸟到管道的数值距离作为状态，而是利用游戏画面的图像作为状态：通过训练一个卷积神经网络模型对游戏状态（即图像）进行判别，预测每个动作的价值（只有两个动作选项，即按键或者不按）。

DQN应用实例—Flappy Bird

主要用到了哪种神经网络模型?



具体训练该智能体的过程可以结合之前13.8小节中介绍的NIPS DQN算法辅助理解。

主观题 10分

设置

65

深度强化学习相对于强化学习有何优势？Nature DQN算法采用哪种神经网络处理图像？

作答

从强化学习到深度强化学习

内容概要

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero

从强化学习到深度强化学习

内容概要

强化学习

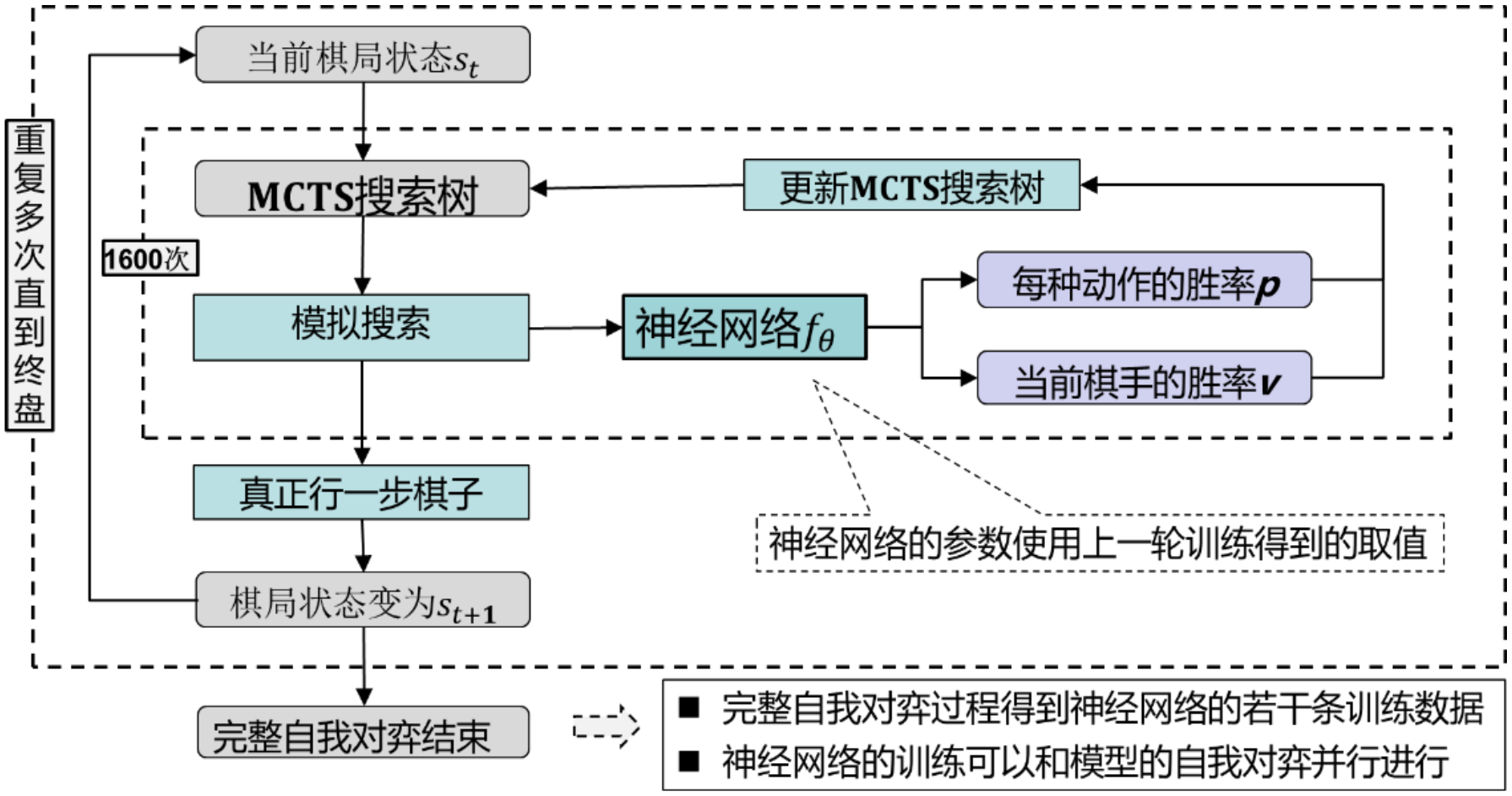
- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- **基于深度强化学习的AI围棋模型AlphaGo Zero**

基于深度强化学习的AI围棋模型AlphaGo Zero

AlphaGo Zero模型整体结构及基本原理



主观题 10分

设置

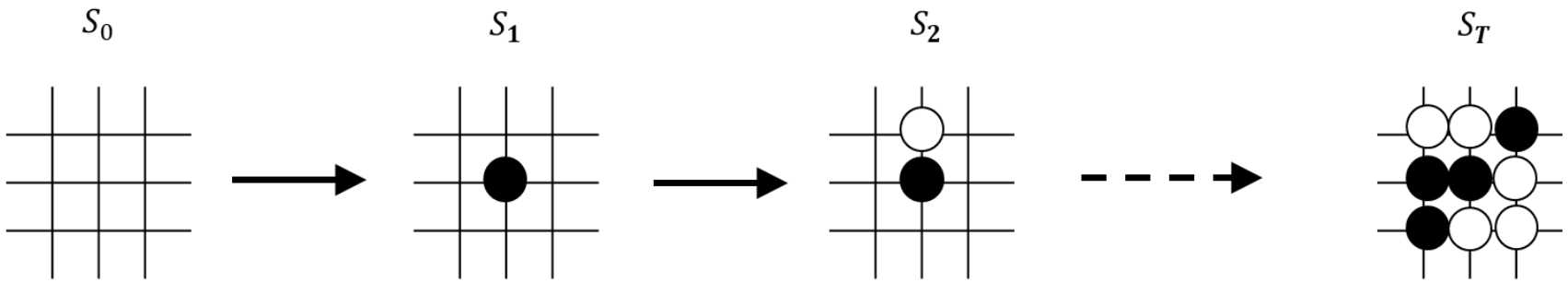
69

请用文字描述一下AlphaGo Zero模型的基本原理，你认为其击败人类围棋冠军的秘诀是什么?

作答

基于深度强化学习的AI围棋模型AlphaGo Zero

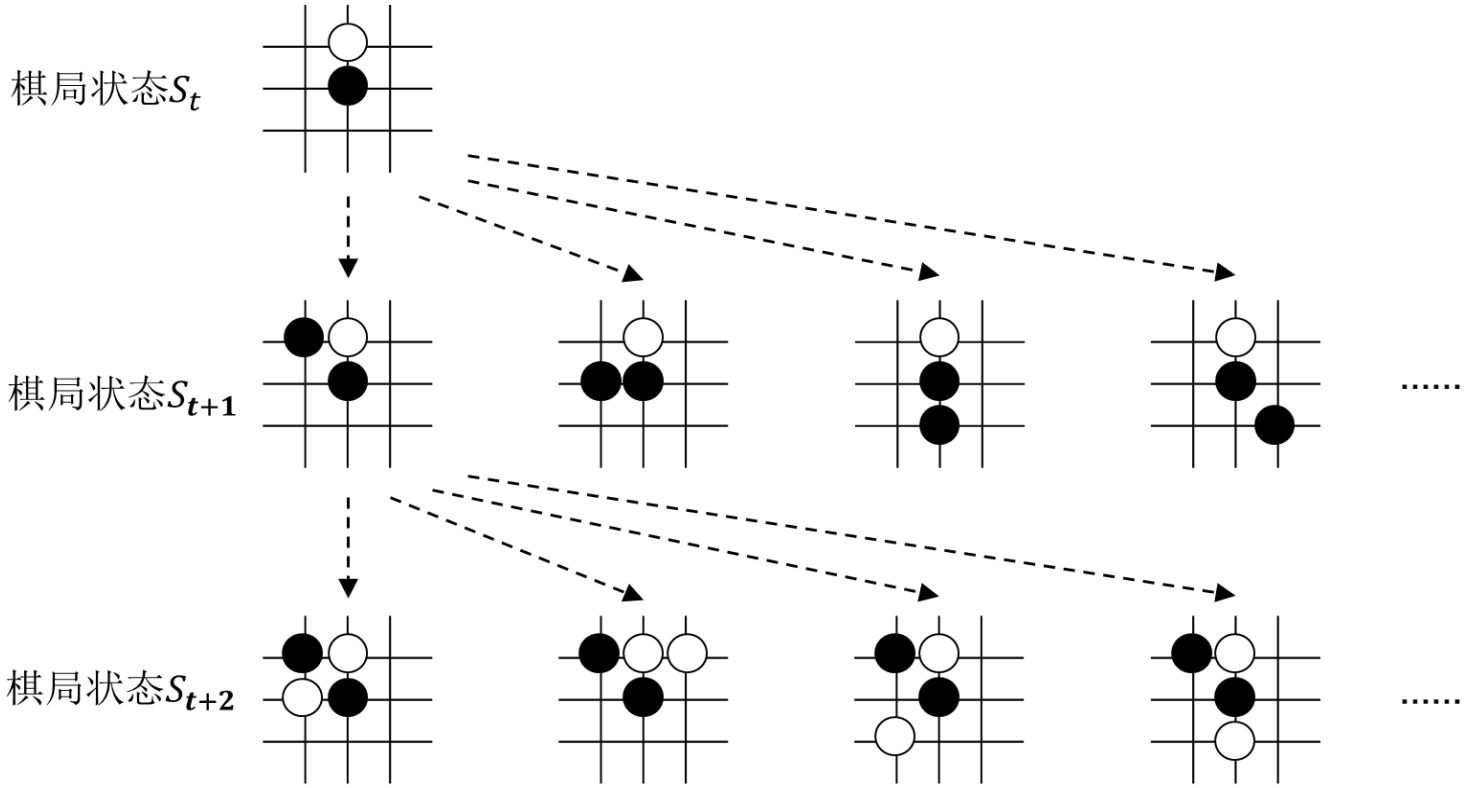
AlphaGo Zero一次完整的自我对弈序列及其样本



AlphaGo Zero一次完整的自我对弈序列及其样本
（$v_0 = v_1 = v_2 = \cdots = v_T = z$，z为该对弈完整序列的终盘最终胜负结果）

AlphaGo Zero的MCTS搜索过程

围棋游戏的状态空间搜索问题



假设从空盘下起，则围棋第一手有361种走法、第二手有360种走法、第三手有359种走法，依次类推....

AlphaGo Zero的MCTS搜索过程

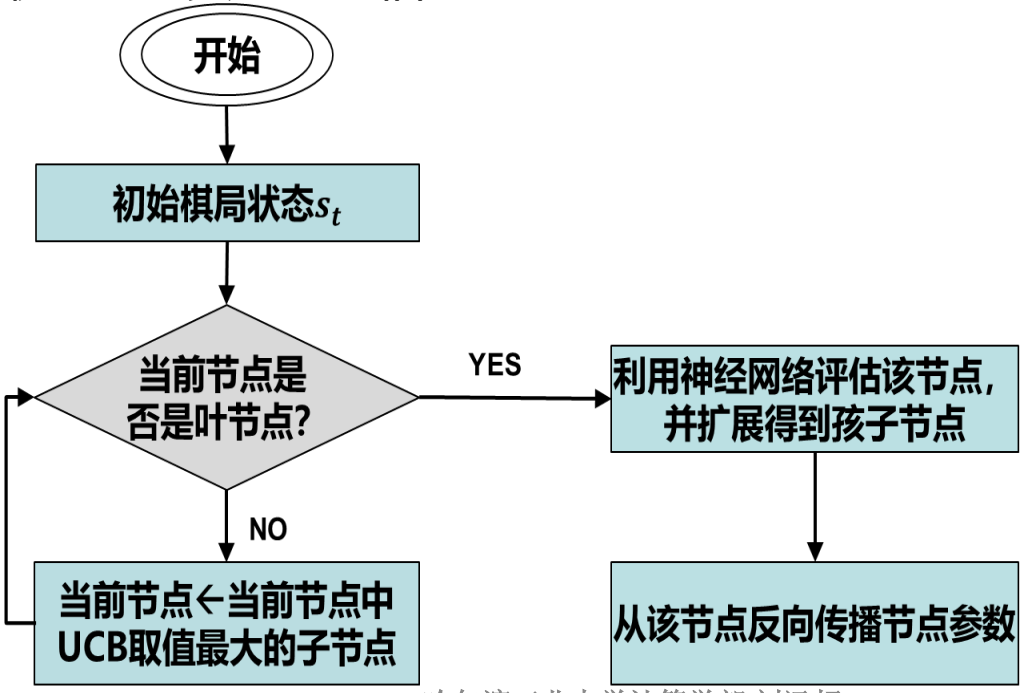
为什么每一步真正落子行棋前进行1600次MCTS模拟搜索?

- 在训练阶段，AlphaGo Zero在每一步真正落子行棋前，面对当前的棋局状态，会进行1600次MCTS模拟搜索，目的是收集以当前棋局状态s_t为根节点的大量棋局及其终盘胜负结果。
- 不同于传统MCTS需要从当前棋局开始将按照某种策略将模拟进行到终盘直到分出胜负的模式，在AlphaGo Zero中，这1600次模拟是通过神经网络来预测的。
- 这种每一步真正落子前，对搜索树模拟多次的做法，很像人类棋手下棋时思考的方式，即每一步落子前模拟了解不同动作的发展方向，并试图根据模拟结果预测最有希望的落子动作。

AlphaGo Zero的MCTS搜索过程

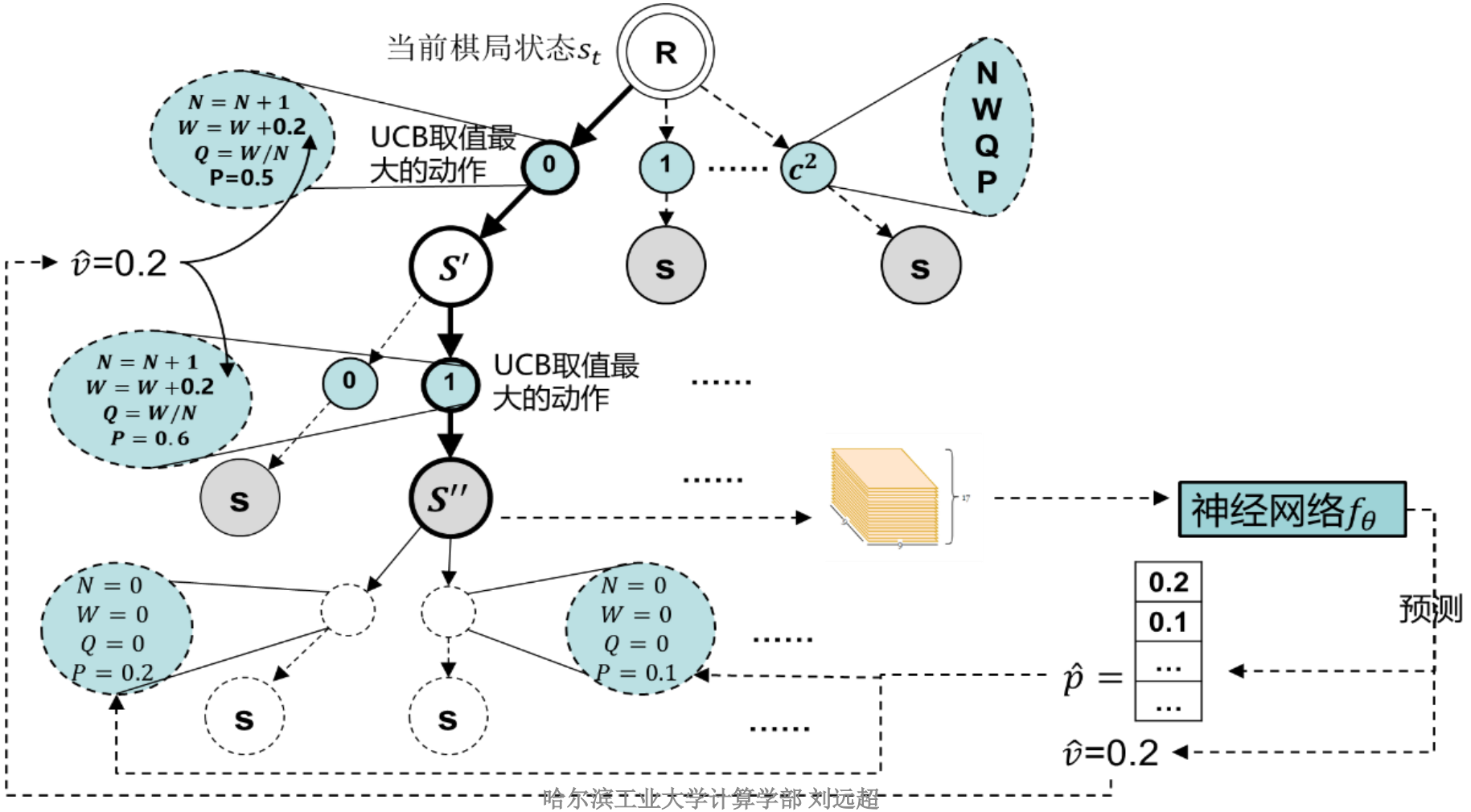
AlphaGo Zero中MCTS每次模拟搜索的流程图

- 1600次模拟搜索的每次都从当前棋局状态s_t作为根节点（即图中13.27中R节点）出发，通过**1）节点选择（多次递归向下）；2）叶节点扩展；3）信息从叶节点向根节点方向的回溯等**基本操作，对搜索树节点进行扩充并更新搜索树中边的信息。
- 搜索树中每个节点保存的实际上是一种棋局，而节点之间的连接则体现了这种转换（走法）最终的胜负概率等信息。



AlphaGo Zero的MCTS搜索过程

AlphaGo Zero中MCTS每次模拟搜索的基本过程



AlphaGo Zero的MCTS搜索过程

下面结合以上两图，对这几种基本步骤进行介绍

1) 节点选择

该步骤的目的。面对当前遇到的棋局状态s_t，和人类棋手的解决思维类似，1600次模拟搜索中的每一次首先要先检索现有“知识库存”中是否有最有可能导致终盘取胜下棋路径，这个“知识库存”在AlphaGo Zero中就是MCTS搜索树，而“较好的路径”需要在该搜索树中通过逐渐向下一步一步地选择节点来完成。

AlphaGo Zero的MCTS搜索过程

UCB计算公式需要用到的几个变量

刚才提到了UCB，在具体介绍UCB的计算公式之前，我们先看一下其中需要用到的几个变量。

在图13.27中，假设当前选择的棋局节点（状态）为s，则对于从s出发可以采取的每个动作节点a_i（图中小圆圈）上将存储四种数值：

1) $N(s,a_i)$：当处于棋局状态s时，选择动作a_i的次数。这个次数是指针对s进行1600次统计中出现的次数；

2) $W(s,a_i)$：当处于棋局状态s时，选择动作a_i产生的总价值。该总价值为利用神经网络预测得到的$\hat{v}$值的和，表示选择动作a_i后终盘获胜的总可能性；

3) $Q(s,a_i)$：当处于棋局状态s时，选择动作a_i产生的平均价值。因此，$Q(s,a_i)=\frac{W(s,a_i)}{N(s,a_i)}$；

4) $P(s,a_i)$：当处于棋局状态s时，选择动作a_i的概率。该项数值是神经网络针对当前状态预测得到的向量$\hat{p}$中与a_i对应的一个元素。如图13.27所示。

AlphaGo Zero的MCTS搜索过程

用于评价并选择动作a_i的UCB值$U(s,a_i)$如何计算：

下面我们看一下当处于棋局状态s时，用于评价并选择动作a_i的UCB值$U(s,a_i)$如何计算：

$$U(s,a_i)=\frac{W(s,a_i)}{N(s,a_i)}+c_{puct}\,P(s,a_i)\frac{\sqrt{\sum_{i=0}^{C^2}N(s,a_i)}}{1+N(s,a_i)} \qquad (13.32)$$

利用分项　　探索分项（惩罚项）

UCB值包括两个分项：

- 第一项是利用（exploitation）项，即$\frac{W(s,a_i)}{N(s,a_i)}$是当处于棋局状态s时，选择动作$a_i$产生的平均价值，其是选择动作$a_i$的胜率估计。
- 第二项是探索项（exploration），其中的$\frac{\sqrt{\sum_{i=0}^{C^2}N(s,a_i)}}{1+N(s,a_i)}$是一个惩罚项，其取值将随着选择动作$a_i$次数的增加而减少。**其目的是引入探索成分，是为了在选择具体动作时除了考虑最大胜率（通过第一分项来体现）外，也能对其它动作进行适当探索**。其中C^2代表棋盘大小（$C=19$）。如果运行1600次模拟，且每次模拟都将选择一个动作，则$\sum_{i=0}^{C^2}N(s,a_i)=1600$。分母加1的目的是为了避免除以0。$P(s,a_i)$的含义之前介绍过。

AlphaGo Zero的MCTS搜索过程

叶节点扩展

- 当MCTS模拟搜索采样过程探索到**非终局的叶子节点状态s_L（意味着该状态没有历史对弈记录可供参考）**时，AlphaGo Zero会利用神经网络对该叶子节点进行预测，得到其各个可能子节点位置上落子（动作a）的概率分布$\hat{p}$和当前行棋方的获胜概率$\hat{v}$。然后创建这些新节点，如图13.27最下方虚线节点所示。该叶子节点s_L到新增加的每个节点k的边上4项信息分别初始化为：

$$\{N(s_L,a)=0,\ W(s_L,a)=0,\ Q(s_L,a)=0,\ P(s_L,a)=\hat{p}[k]\} \quad (13.33)$$

- 可见，新加入的动作节点中存储的N、W、Q值均初始化为0，而P值将通过神经网络获得预测值，其为向量 $\hat{p}$ 的第k个元素。

AlphaGo Zero的MCTS搜索过程

回溯（backpropagation）

回溯（backpropagation）

这一步从刚才扩展的叶子节点开始将如公式（13.33）所示的新信息，向根节点逐层回溯，即将新得到的信息回溯累加到其各个祖先节点的分支上。其具体公式如下：

$$N(s_t, a_t) = N(s_t, a_t) + 1 \tag{13.34}$$

$$W(s_t, a_t) = W(s_t, a_t) + \hat{v} \tag{13.35}$$

$$Q(s_t, a_t) = \frac{W(s_t, a_t)}{N(s_t, a_t)} \tag{13.36}$$

其中$\hat{v}$值需要通过神经网络预测得到，如图13.27右侧所示。

上述回溯过程可以从图13.27中自下而上的动作节点信息列表（N、W、Q、P）取值中可以看到。

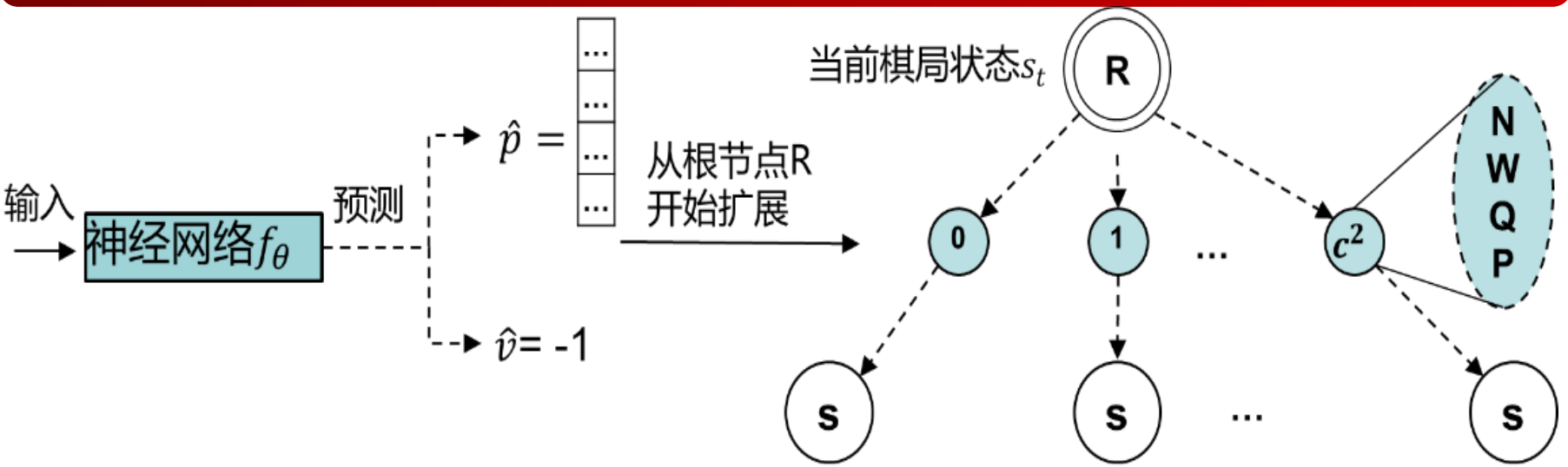
从空盘棋局状态开始MCTS算法的模拟示例

举例

- 以上介绍了1600次MCTS模拟树每次模拟搜索的一般过程。
- **下面通过举例：**
  - 从初始空盘棋局状态开始，通过描述MCTS搜索树的第1次模拟搜索、第2次模拟搜索、第 j 次一般性模拟搜索的实际计算过程来获得更为具体的感性认识。
  - 然后再介绍1600次MCTS模拟搜索结束后具体行棋落子所采取的策略。

从空盘棋局状态开始MCTS算法的模拟示例

第 1 次模拟示例



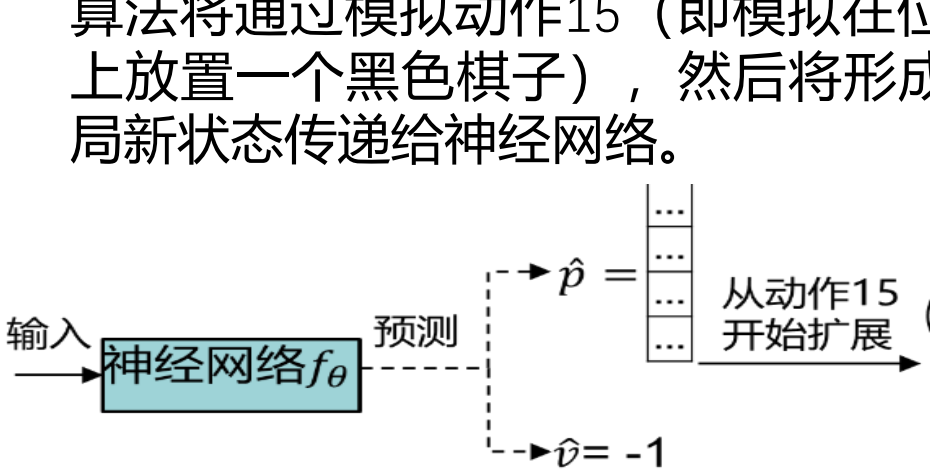
从空盘开始MCTS树搜索的第一次模拟示例

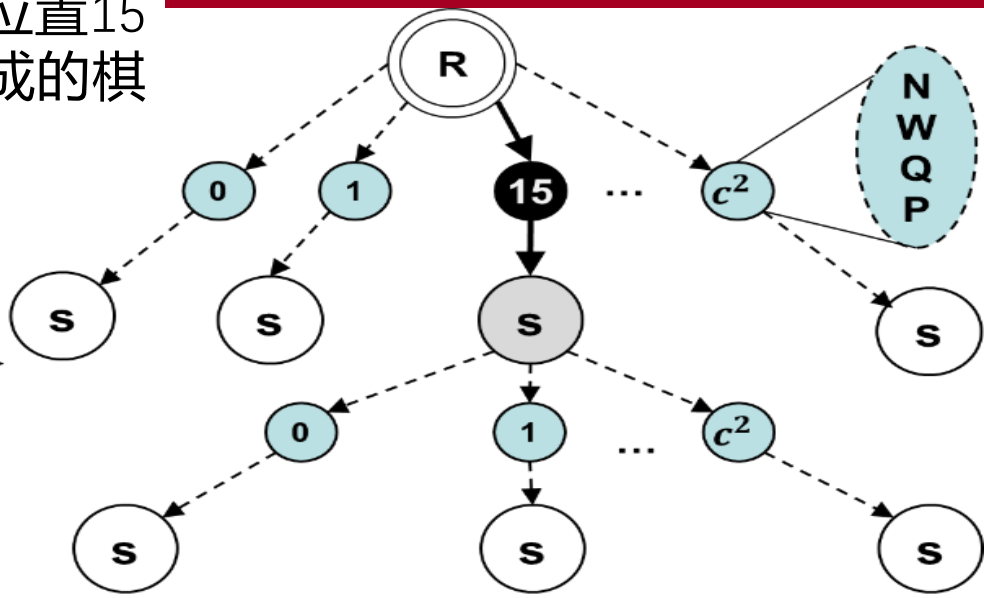
- 如图所示，假设从初始空棋盘状态开始，此时神经网络的输入为当前棋局状态、之前的15个棋局状态（空盘）以及一个额外的19×19矩阵（由于围棋第一个下子的是黑方，所以其元素全为0，如果白方则全为1）。
- 因此神经网络的输入为19×19×17的张量。其输出将为一个包含19×19+1个元素的向量$\hat{p}$，其含义是基于当前棋局状态选择每个动作的概率（包括“pass”）。神经网络还将输出一个在(−1，1)范围内的浮点值$\hat{v}$，含义是当前行棋方最终的胜率。

哈尔滨工业大学计算学部 刘远超

第 2 次模拟示例

- 假设其中UCB取值最大的动作是15，因此算法将通过模拟动作15（即模拟在位置15上放置一个黑色棋子），然后将形成的棋局新状态传递给神经网络。





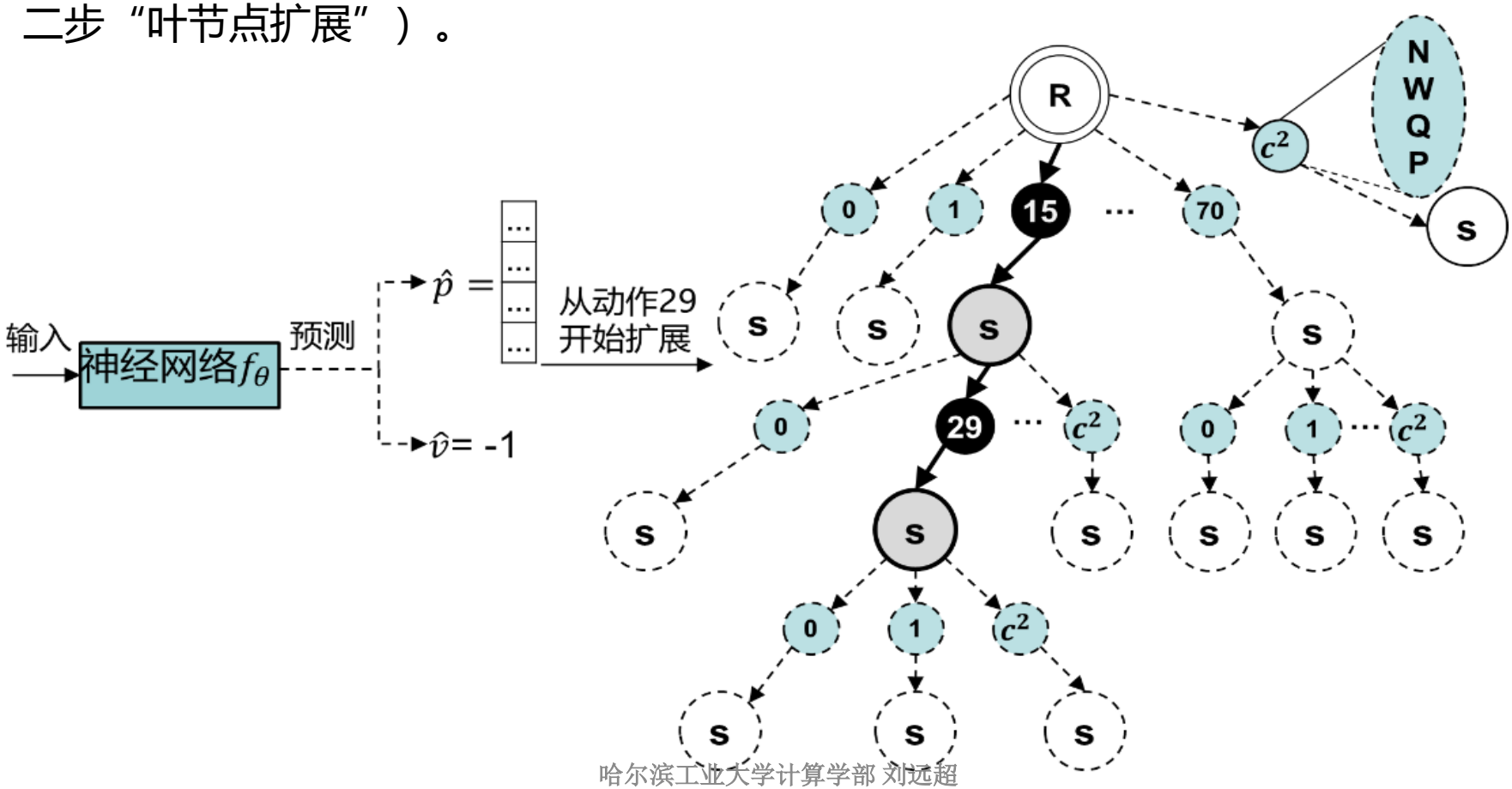
结合神经网络的返回值（假设返回$v(s,a_{15})=-1$），选择动作15以后，该动作节点上存储的信息为：

1) $N(s,a_{15})=N(s,a_{15})+1=0+1=1$。其加1是因为刚才选择了一次。
2) $W(s,a_{15})=W(s,a_{15})+v(s,a_{15})=0+(-1)=-1$。因为本例中从空盘开始，$N(s,a_i)=W(s,a_i)=0$，$v(s,a_{15})$为通过神经网络预测得到。
3) $Q(s,a_{15})=W(s,a_{15})/N(s,a_{15})=-1/1=-1$。
4) $P(s,a_{15})$：该项数值是神经网络针对当前状态s预测得到的向量$\hat{p}$中与a_{15}对应的一个元素。

这些值将通过节点15之前被访问过的节点向根节点方向反向传播。由于节点15之前唯一访问过的节点是根节点，因此只需要将这些值反向传播到根节点。

第 j 次模拟示例

- 1600次中的第j次迭代，将仍然从根节点R开始，对于根节点R，计算每个动作a_i的$U(r,a_i)$并比较，假设其中取得最大值的仍然是动作15。但由于我们在第二次模拟搜索中已经扩展过节点15，因此，在第j次模拟中我们需要将计算从s_{15}开始的每个动作的UCB取值$U(s_{15},a_i)$。
- 假设其中取值最大的是动作29，且假设动作29在第j次迭代之前没有被访问过的叶节点，因此需要在这里再次使用神经网络评估此处的棋局状态，以扩展节点29（即第二步“叶节点扩展”）。



从空盘棋局状态开始MCTS算法的模拟示例

“回溯阶段”

- 之后，将进入“回溯阶段”，即动作节点上的数值将更新并向根节点方向再次反向传播。结合神经网络的返回值（仍然假设$v(s,a_{29})=-1$），选择动作29以后，该动作节点上存储的信息为：
  - $N(s,a_{29})=N(s,a_{29})+1=0+1=1$。其加1是因为刚才选择了一次。
  - $W(s,a_{29})=W(s,a_{29})+v(s,a_{29})=0+(-1)=-1$。因为本例中从空盘开始，$N(s,a_{29})=W(s,a_{29})=0$，$v(s,a_{29})$为通过神经网络预测得到。
  - $Q(s,a_{29})=W(s,a_{29})/N(s,a_{29})=-1/1=-1$
  - $P(s,a_{29})$：该项数值是神经网络针对当前状态s预测得到的向量$\hat{p}$中与a_{29}对应的一个元素。

从空盘棋局状态开始MCTS算法的模拟示例

然后向后（根节点方向）传播

然后向后传播，则动作节点15上存储的信息更新为：

1) $N(s,a_{15}) = N(s,a_{15}) + 1 = 1 + 1 = 2$。回忆一下，该节点之前已经访问过了一次，且假设只有一次。

2) $W(s,a_{15}) = W(s,a_{15}) + v(s,a_{29}) = (-1) + (-1) = -2$

3) $Q(s,a_{15}) = W(s,a_{15})/N(s,a_{15}) = -2/2 = -1$

4) $P(s,a_{15})$：该项数值是神经网络针对当前状态s预测得到的向量$\hat{p}$中与a_{15}对应的一个元素。

然后继续反向传播，则根节点上的信息也做类似更新（略）。

以上模拟搜索过程将进行1600次（默认值）。

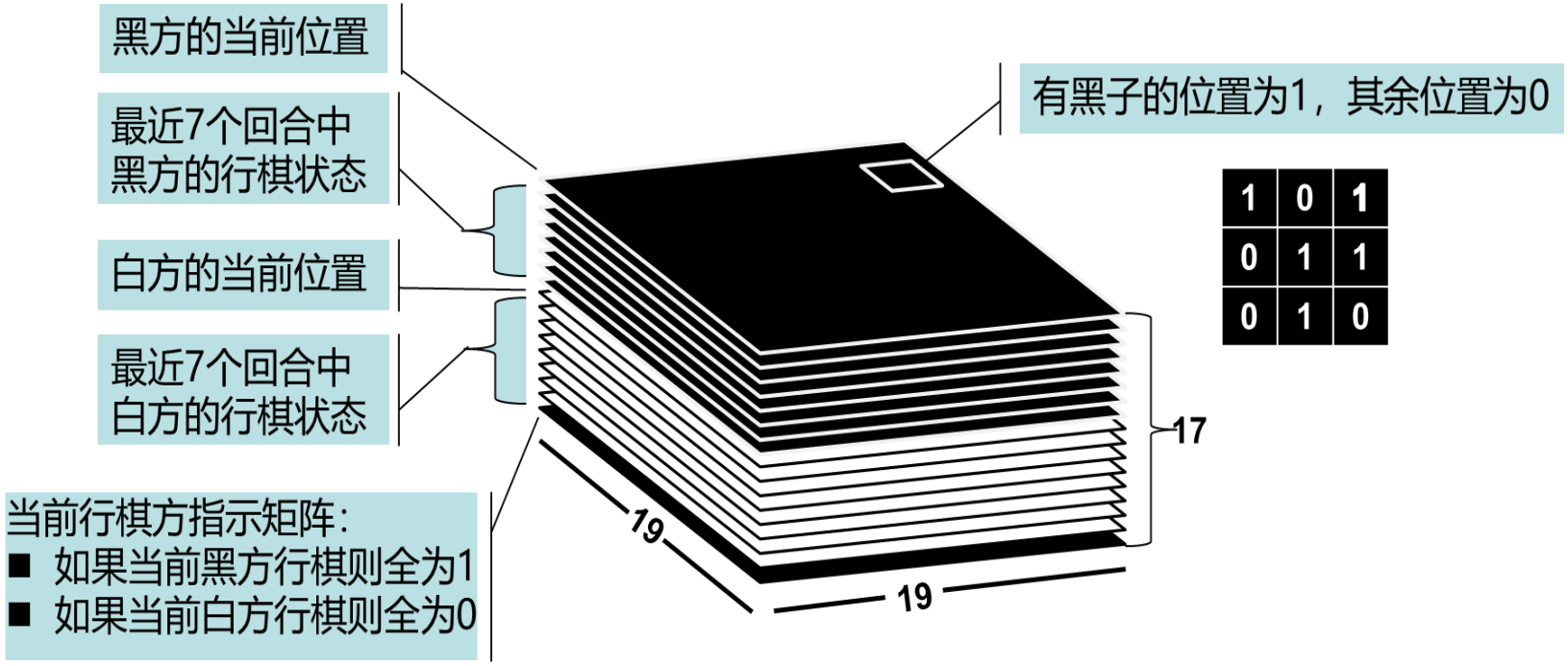
从空盘棋局状态开始MCTS算法的模拟示例

1600次模拟结束后执行具体落子动作的策略

- 1600次模拟结束后，将从当前棋局状态即图中的根节点R出发，根据如下策略选择行棋的MCTS分支选择一个动作，并真正落子（该落子属于当前轮次自我对弈中的一次落子）：
  - $$\pi(a|s) = \frac{N(s_t,a)^{1/\tau}}{\sum_b N(s_t,b)^{1/\tau}} \quad (13.35)$$
    - 其中，s_t为当前的根节点，即为图13.27到图13.30中的节点R。τ为温度参数，用于控制探索的程度。**每次完整自我对弈的前30步（不是MCTS模拟的前30次）一般设为τ=1，以加强早期的探索**。对于剩下的其它步，τ的取值将逐渐降低并趋近于0，表示倾向于选择访问次数最多的子节点。
    - 选中并（真正）执行某动作后，**将进入新的状态。与该实际动作对应的新状态将成为新的根节点R**。然后重复上述自我对弈以及16000次模拟的过程，进行训练学习。

AlphaGo Zero中的神经网络

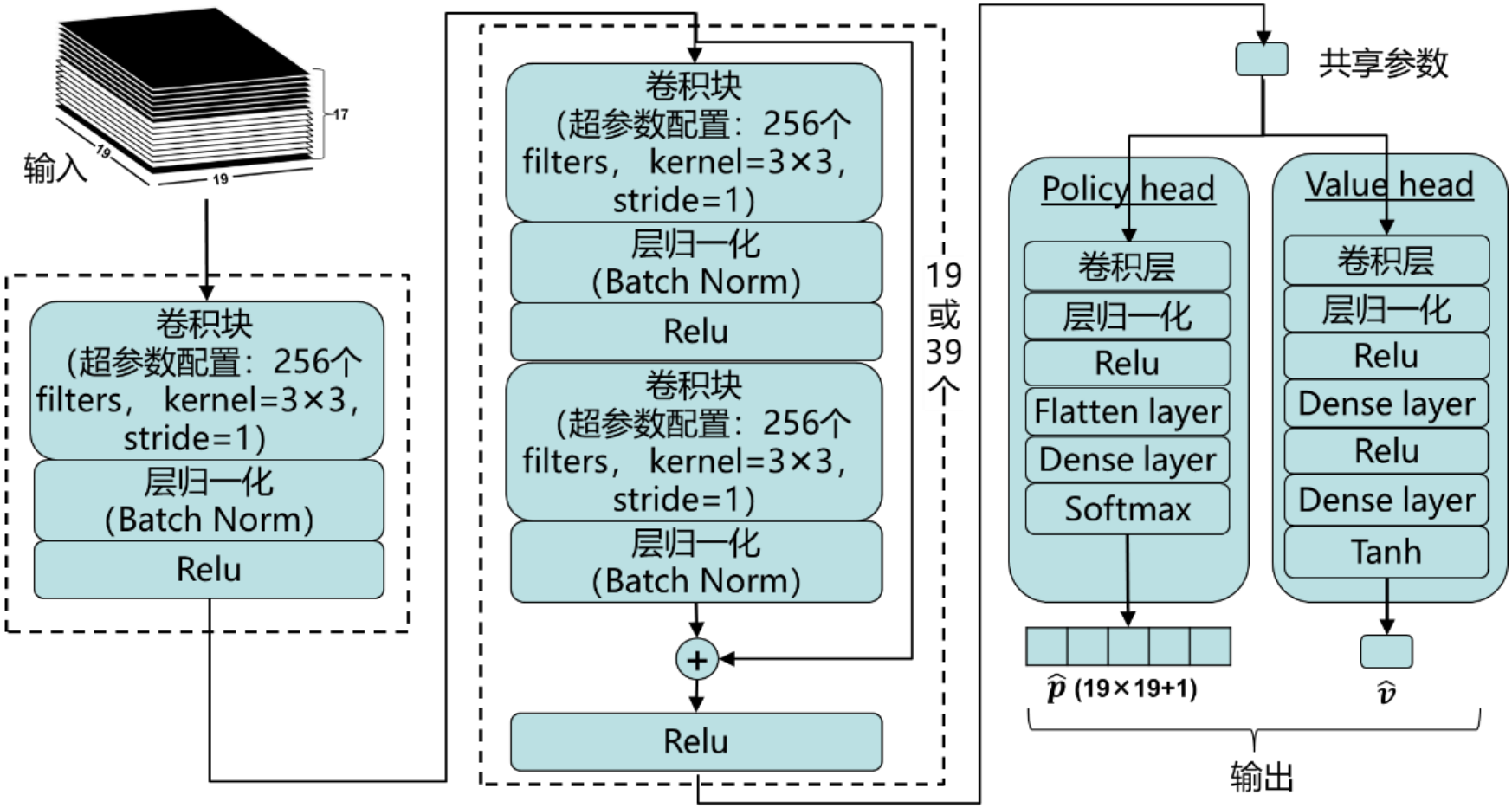
AlphaGo Zero中神经网络的输入、模型框架及输出。



AlphaGo Zero中神经网络的输入

AlphaGo Zero中的神经网络

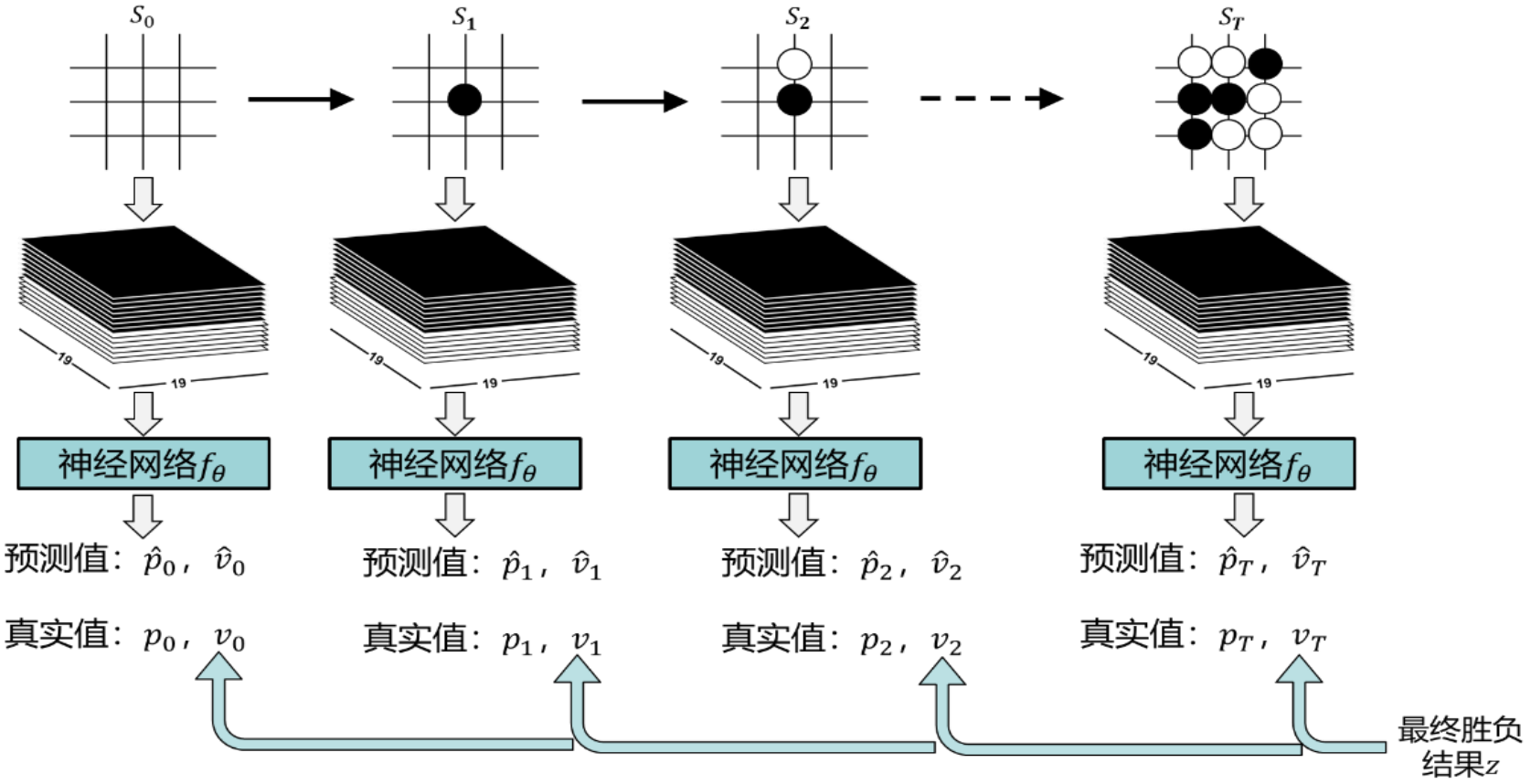
AlphaGo Zero神经网络的结构



AlphaGo Zero神经网络的结构

AlphaGo Zero中的神经网络

Alpha Zero的神经网络是怎样训练的



Alpha Zero中的神经网络训练的数据样本来源

AlphaGo Zero中的神经网络

神经网络的训练目标

- 神经网络的训练目标就是要尽可能的缩小这两个预测值与各自真实值之间的差距。因此损失函数L可以表示如下：
  - $L = (\hat{v} - z)^2 - \hat{p}^T log(p) + c||\theta||^2$ (13.36)
- 即损失函数由三部分组成：
  - 第一部分用**均方误差损失**来评估**神经网络预测的胜负结果**$\hat{v}$和真实胜负结果$v = z$之间的差异；
  - 第二部分用交叉熵损失来评估**神经网络输出的概率分布向量**$\hat{p}$与MCTS搜索得到的概率向量p之间的差异；
  - 第三部分是参数θ的**L2正则化项**，c为正则系数超参数。
- 通过使用梯度下降法，经过若干轮次迭代，使该损失函数数值减少，可以得到经过优化的新参数取值，**并用来指导下一轮迭代中MCTS搜索**，从而使对弈水平逐渐提高。

从强化学习到深度强化学习

本章小结

强化学习

- 强化学习的引入
- 强化学习的要素
- 探索-利用困境
- 问题的简化：马尔可夫决策过程与贝尔曼方程
- 序贯决策问题与强化学习
- 使用蒙特卡罗法求解强化学习
- 时序差分法（TD）

深度强化学习

- 价值函数的近似表示
- NIPS DQN
- NATURE DQN
- DQN典型应用案例FlappyBird
- 基于深度强化学习的AI围棋模型AlphaGo Zero